MINI PROJECT | UMESH KUMBHARKAR | INTERMEDIATE

CLOUDFLIX

Netflix-Style Landing Page Using Jenkins CI/CD

Step-by-step illustrated documentation: from GitHub to Docker Hub to a live container on AWS EC2

Freestyle job GitHub Docker Hub AWS EC2

Document	Details
Project	Netflix-style UI CI/CD pipeline by Umesh Kumbharkar
Difficulty	Intermediate
Technology	AWS EC2 (Amazon Linux 2023), Jenkins (Freestyle project), Git and GitHub, Docker and Docker Hub, Nginx
Version / date	1.0	October 19, 2026
Figures	42 numbered figures (diagrams, screens and terminal sessions) in step order

Prepared by Umesh Kumbharkar

LinkedIn: www.linkedin.com/in/umesh-kumbharkar

This is a learning project with an original Netflix-inspired design. It is not the actual Netflix application or its proprietary source code.

1. Project overview

In this project you build a Netflix-inspired landing page called **CloudFlix**, keep its source in GitHub, and use a **Jenkins Freestyle project** to build a Docker image, test it, push it to **Docker Hub**, and run it as a container on your own **AWS EC2** instance. The guide starts from launching the EC2 instance and installing Jenkins, so no prior setup is needed.

The big picture

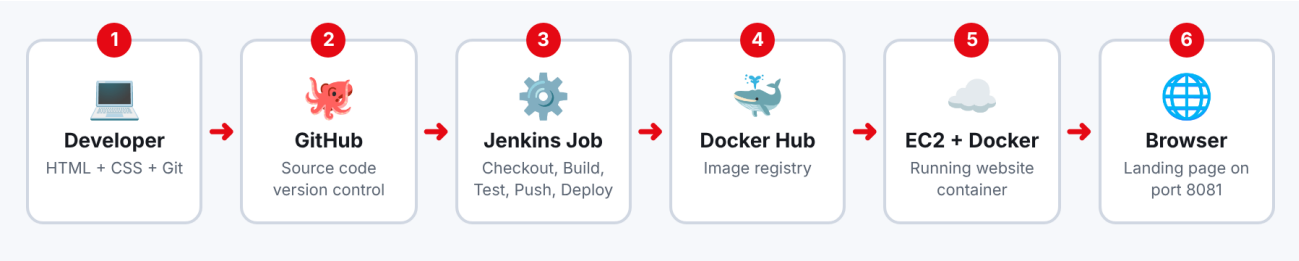


Figure 1. *End-to-end flow: developer to GitHub to Jenkins to Docker Hub to EC2 to browser*

What you will learn

Skill	Skill
Launch and configure an AWS EC2 instance	Install Jenkins and access its web UI
Install Git, Docker and the required plugins	Create a GitHub repository and push the website source
Create a Jenkins Freestyle project	Build and test a Docker image
Authenticate securely with Docker Hub and push the image	Deploy the image as a container and view the landing page

Before you start

- An **AWS account** with permission to launch EC2 instances and edit security groups.
- A **GitHub account** and a **Docker Hub account** (both free tiers are enough).
- A computer with **Git** installed and an SSH client (or use EC2 Instance Connect from the AWS console).
- A text editor such as VS Code to create `index.html` and `Dockerfile`.

Your 10-phase roadmap

Phase	Title	Steps	Outcome
1	Launch an AWS EC2 instance	1 to 3	A running Amazon Linux 2023 server with a locked-down firewall
2	Connect to EC2 and install Jenkins	4 to 9	Jenkins dashboard reachable on port 8080
3	Install Git and Docker on EC2	10 to 12	Jenkins can run Docker commands
4	Create the GitHub repository	13 to 17	`index.html` and `Dockerfile` on the `main` branch
5	Prepare Docker Hub	18 to 19	Public repository and a push access token
6	Configure Jenkins credentials and plugins	20 to 21	Credential `dockerhub-creds` stored in Jenkins
7	Create the Jenkins Freestyle job	22 to 25	Job wired to GitHub with Docker Hub secrets bound

Phase	Title	Steps	Outcome
8	Add the CI/CD build script	26 to 27	Six-stage Execute shell script saved
9	Run and verify the pipeline	28 to 30	Image on Docker Hub and the site live on port 8081
10	Test automatic CI/CD behaviour	n/a	Change, push, rebuild and redeploy loop

How to read this guide

- Every step has the commands or settings to use, followed by a numbered **figure** showing what the screen should look like.
- **Red numbered badges** on a figure match the numbered instructions in the text.
- Replace placeholders such as `YOUR_EC2_PUBLIC_IP`, `YOUR-GITHUB-USERNAME` and `YOUR-DOCKERHUB-USERNAME` with your own values (see Appendix A).
- Run commands one at a time and confirm the expected result before moving on.

> **About the screenshots**
>
> The figures are illustrative reproductions of each screen and terminal, drawn to match the steps in this guide. Cloud consoles and Jenkins change their layout over time, and values such as IP addresses, instance IDs and version numbers will differ on your account. The CloudFlix page in Figure 38 is rendered from the real `index.html` below.
>
> If you want to document your own run, take screenshots at the same points and replace the figures one for one.

PHASE 1 Launch an AWS EC2 instance

Steps 1 to 3

STEP 1 Open the EC2 console

1. Log in to the **AWS Management Console**.
2. Search for **EC2**.
3. Select **Instances**, then **Launch instances**.

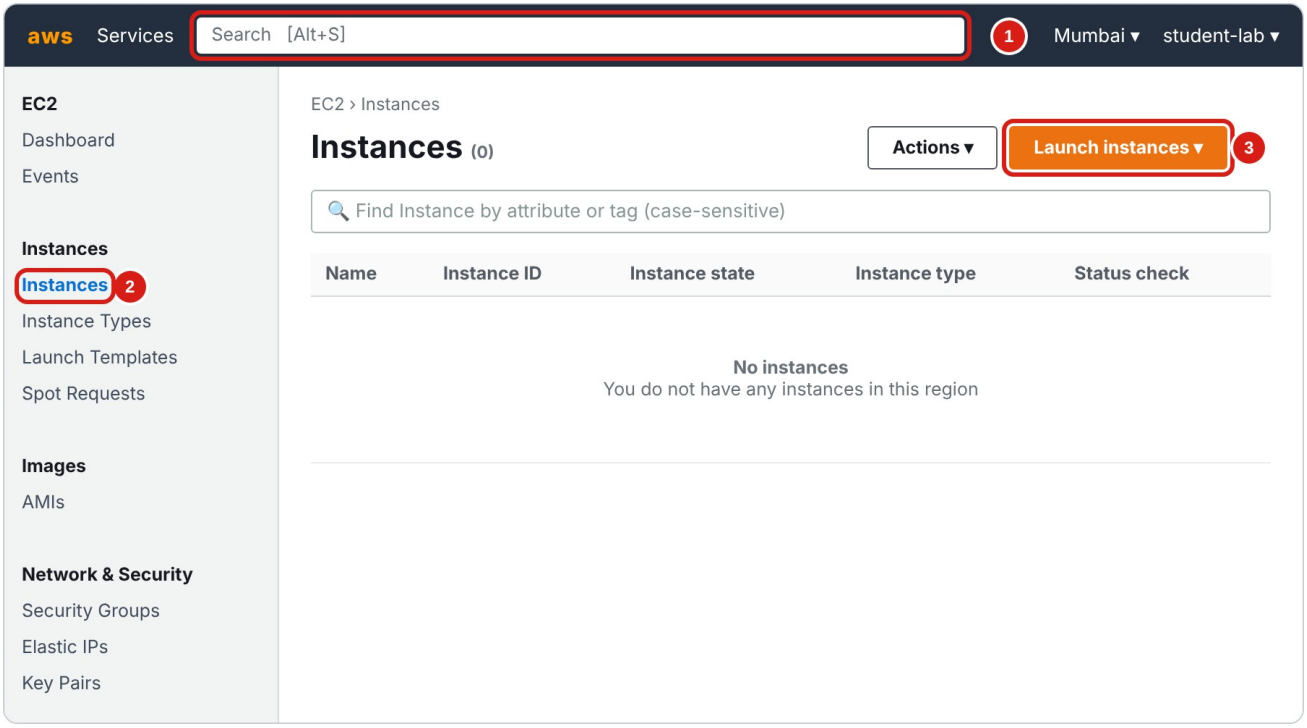


Figure 2. *EC2 console: search for EC2, open Instances, then choose Launch instances* (illustrative screen)

STEP 2 Configure the instance

Use this lab configuration:

Setting	Value
Name	`CloudMessenger-Jenkins-CICD`
AMI	Amazon Linux 2023
Instance type	`t3.medium` recommended for this combined lab
Key pair	Create or select a key pair
Storage	20 to 30 GiB recommended
Network	Use a VPC and subnet with outbound internet access

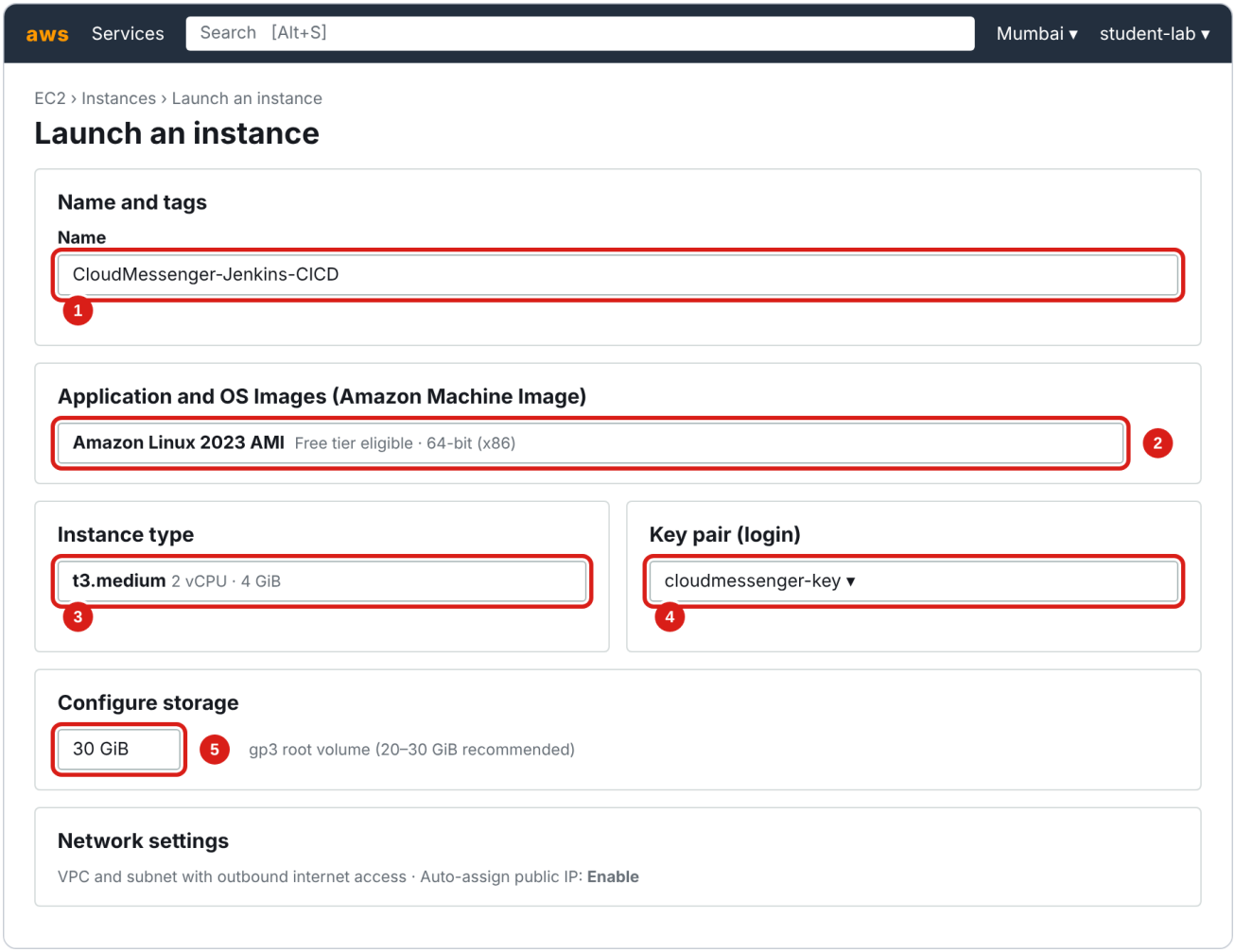


Figure 3. *Launch an instance: name, AMI, instance type, key pair and storage* (illustrative screen)

> **Sizing**
> Jenkins and Docker both run on this machine, so allow enough memory and disk space for building images.

STEP 3 Configure the security group

Create a security group named `Jenkins-CICD-SG` with these inbound rules:

Port	Type	Source and purpose
22	SSH	Source: My IP
8080	Custom TCP	Source: My IP. Jenkins UI
8081	Custom TCP	Source: My IP. Website output

Port 8080 is for Jenkins; port 8081 serves your containerized website. AWS security groups act as instance-level firewalls.

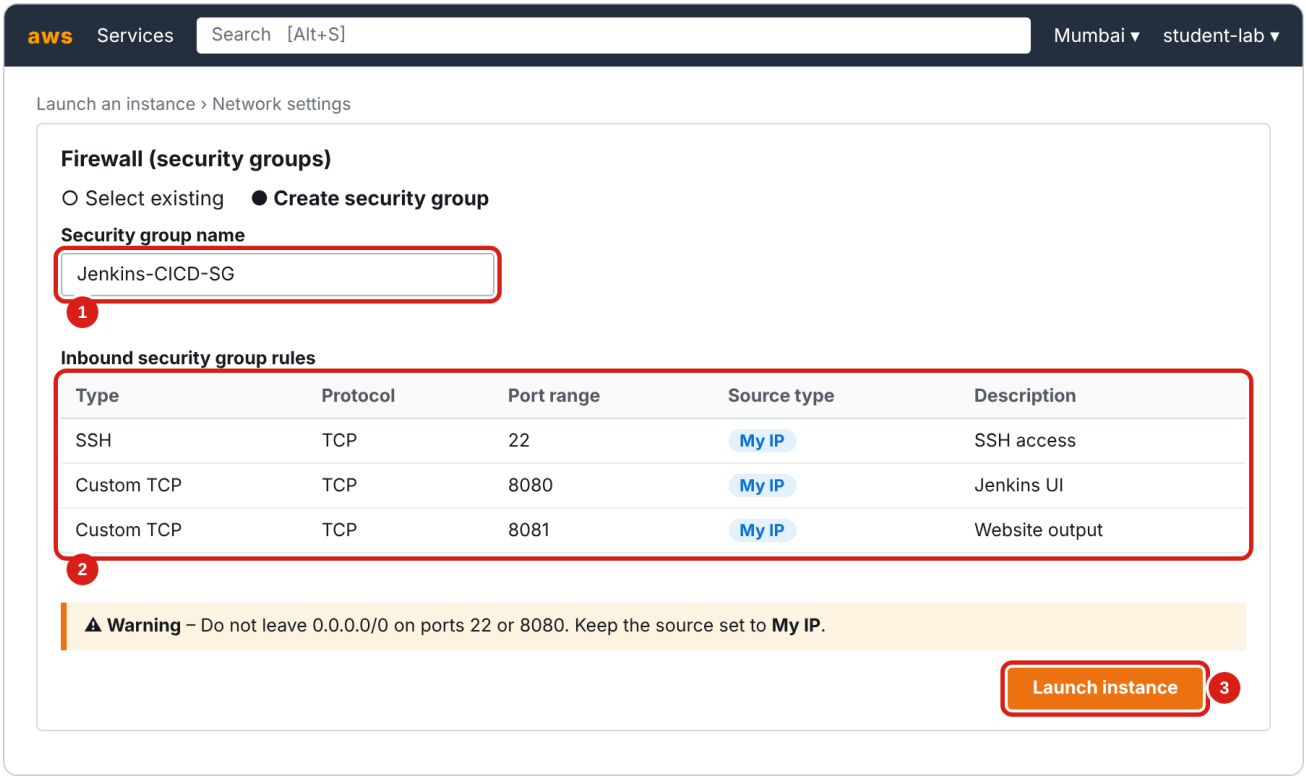


Figure 4. *Security group Jenkins-CICD-SG with ports 22, 8080 and 8081 limited to My IP* (illustrative screen)

> **Do not expose Jenkins or SSH to the whole internet**
> Restrict access to your own public IP for this lab.

1. Click **Launch instance**.
2. Wait until the state is **Running** and the status checks pass.
3. Copy the **Public IPv4 address**. You will use it as `YOUR_EC2_PUBLIC_IP` throughout this guide.

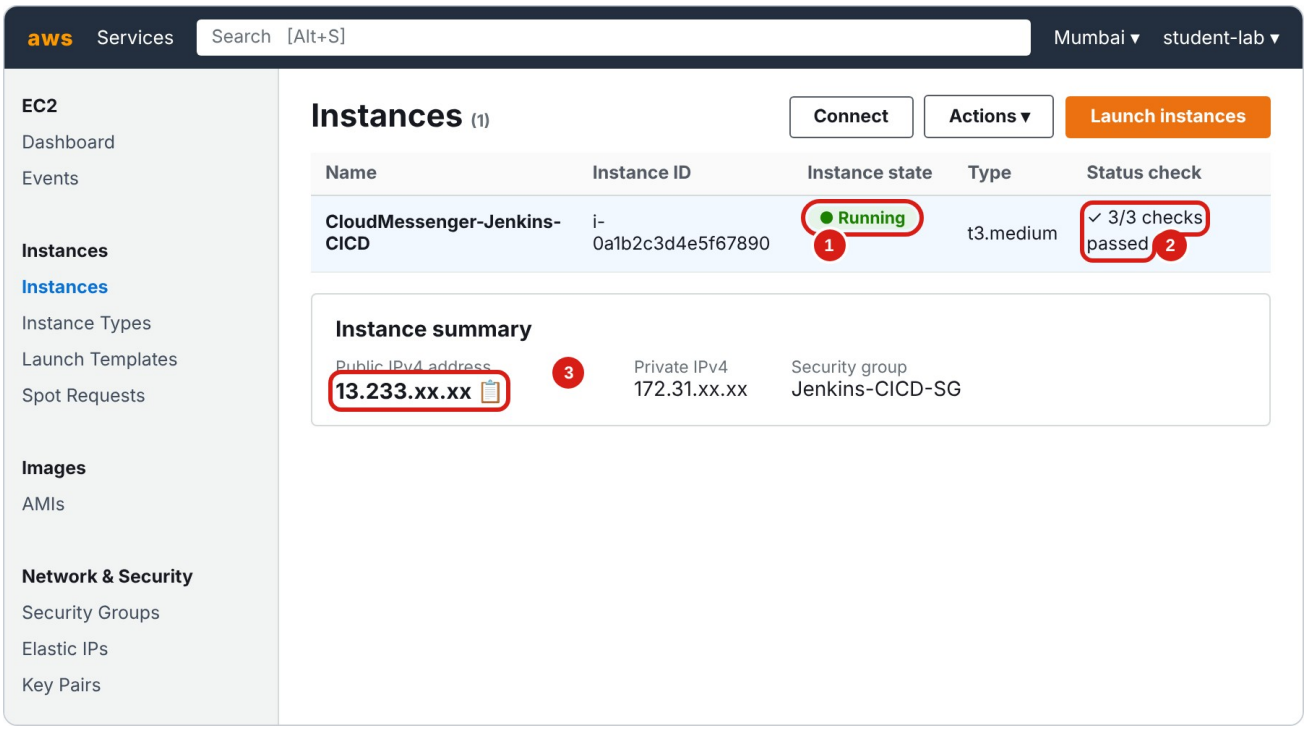


Figure 5. *Instance running with status checks passed and the Public IPv4 address visible* (illustrative screen)

PHASE 2 Connect to EC2 and install Jenkins

Steps 4 to 9

STEP 4 Connect to Amazon Linux

Select the instance and click **Connect**, then **EC2 Instance Connect** if available.

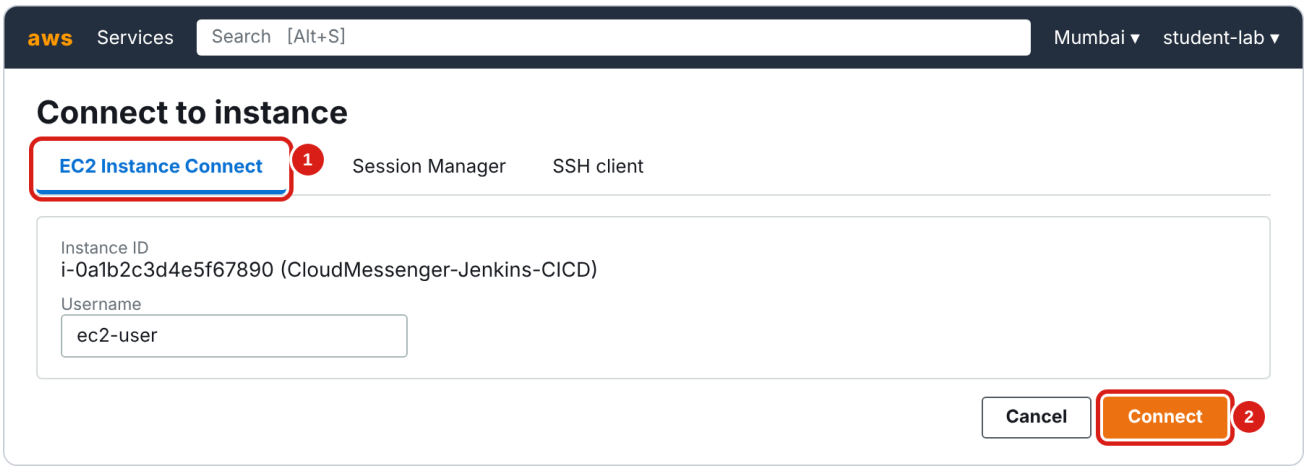


Figure 6. *Connect to instance using the EC2 Instance Connect tab* (illustrative screen)

Alternatively, use SSH from your terminal. Replace the key filename and IP with your own values:

```
ssh -i your-key.pem ec2-user@YOUR_EC2_PUBLIC_IP
```

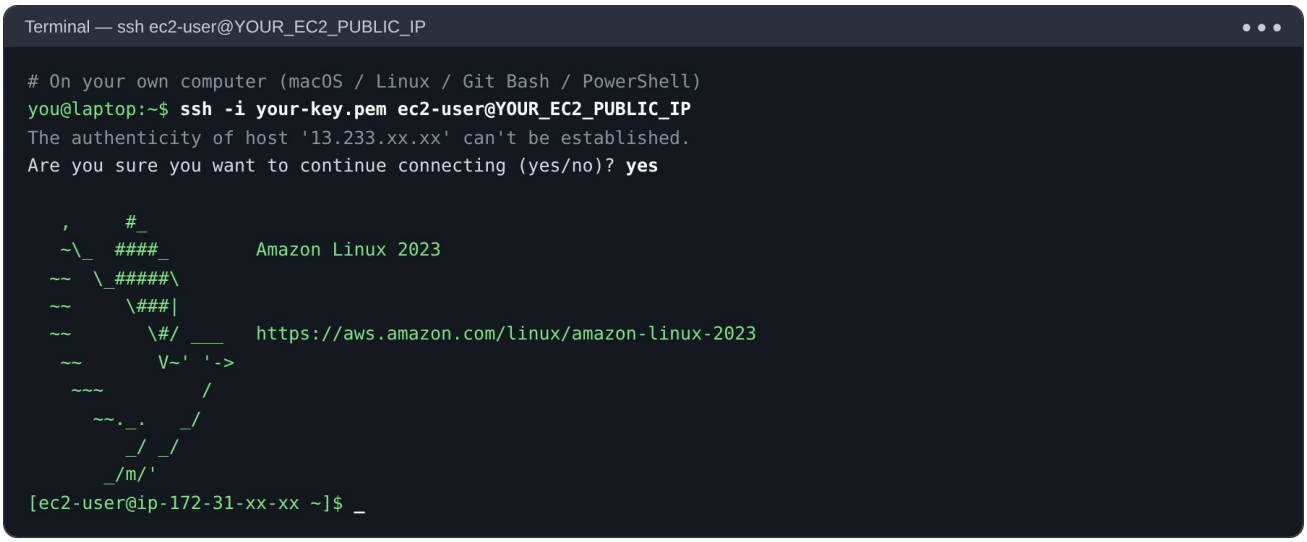


Figure 7. *Successful SSH login showing the Amazon Linux 2023 banner and prompt* (illustrative screen)

STEP 5 Update packages

```
sudo dnf update -y
```

```
ec2-user@ip-172-31-xx-xx:~
[ec2-user@ip-172-31 ~]$ sudo dnf update -y
Amazon Linux 2023 repository                34 MB/s |  41 MB  00:01
Last metadata expiration check: 0:00:03 ago.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31 ~]$
```

Figure 8. *Package update completes* (illustrative screen)

STEP 6 Install Java 21

Jenkins requires a compatible Java runtime. We use Amazon Corretto 21.

```
sudo dnf install -y java-21-amazon-corretto
java -version
```

Expected result: `java -version` shows Java 21 in the output.

```
ec2-user@ip-172-31-xx-xx:~
[ec2-user@ip-172-31 ~]$ sudo dnf install -y java-21-amazon-corretto
... Installing: java-21-amazon-corretto ...
Complete!

[ec2-user@ip-172-31 ~]$ java -version
openjdk version "21.0.x" 2025-xx-xx LTS
OpenJDK Runtime Environment Corretto-21.0.x (build 21.0.x+x-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.x (build 21.0.x+x-LTS, mixed mode, sharing)
↑ the output must mention version 21
```

Figure 9. *Java 21 (Amazon Corretto) installed and verified* (illustrative screen)

STEP 7 Add the Jenkins repository

Run each command separately. First add the repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
  https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

Import the repository signing key:

```
sudo rpm --import \
  https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

Install Jenkins:

```
sudo dnf install -y jenkins
```

```
ec2-user@ip-172-31-xx-xx:~
[ec2-user@ip-172-31 ~]$ sudo wget -O /etc/yum.repos.d/jenkins.repo \
    https://pkg.jenkins.io/rpm-stable/jenkins.repo
'/etc/yum.repos.d/jenkins.repo' saved [85/85]

[ec2-user@ip-172-31 ~]$ sudo rpm --import \
    https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key

[ec2-user@ip-172-31 ~]$ sudo dnf install -y jenkins
Installing:  jenkins   noarch   2.xxx.x-1.1   jenkins   ~90 M
Complete!
```

Figure 10. *Jenkins repository added, key imported and Jenkins installed* (illustrative screen)

STEP 8 Start Jenkins

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

Expected result: The service shows **active (running)** before you continue.

```
ec2-user@ip-172-31-xx-xx:~
[ec2-user@ip-172-31 ~]$ sudo systemctl enable jenkins
[ec2-user@ip-172-31 ~]$ sudo systemctl start jenkins
[ec2-user@ip-172-31 ~]$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled)
     Active: active (running) since Mon 2026-10-19 11:42:07 UTC; 9s ago
   Main PID: 28431 (java)
      Tasks: 45 (limit: 4545)
Press q to exit the status view
```

Figure 11. *Jenkins service enabled and active (running)* (illustrative screen)

STEP 9 Open the Jenkins UI

In your browser, open `http://YOUR_EC2_PUBLIC_IP:8080`. You should see the Jenkins unlock screen. Get the initial password from your EC2 terminal:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

```
ec2-user@ip-172-31-xx-xx:~
[ec2-user@ip-172-31 ~]$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
3f9c1a7e2b4d4c8f9a6e5d1b0c7a2e84
↑ copy this value into the Administrator password box
```

Figure 12. *Reading the initial administrator password in the EC2 terminal* (illustrative screen)

Copy the password into Jenkins, then follow this sequence: **Continue**, **Install suggested plugins**, **Create admin account**.

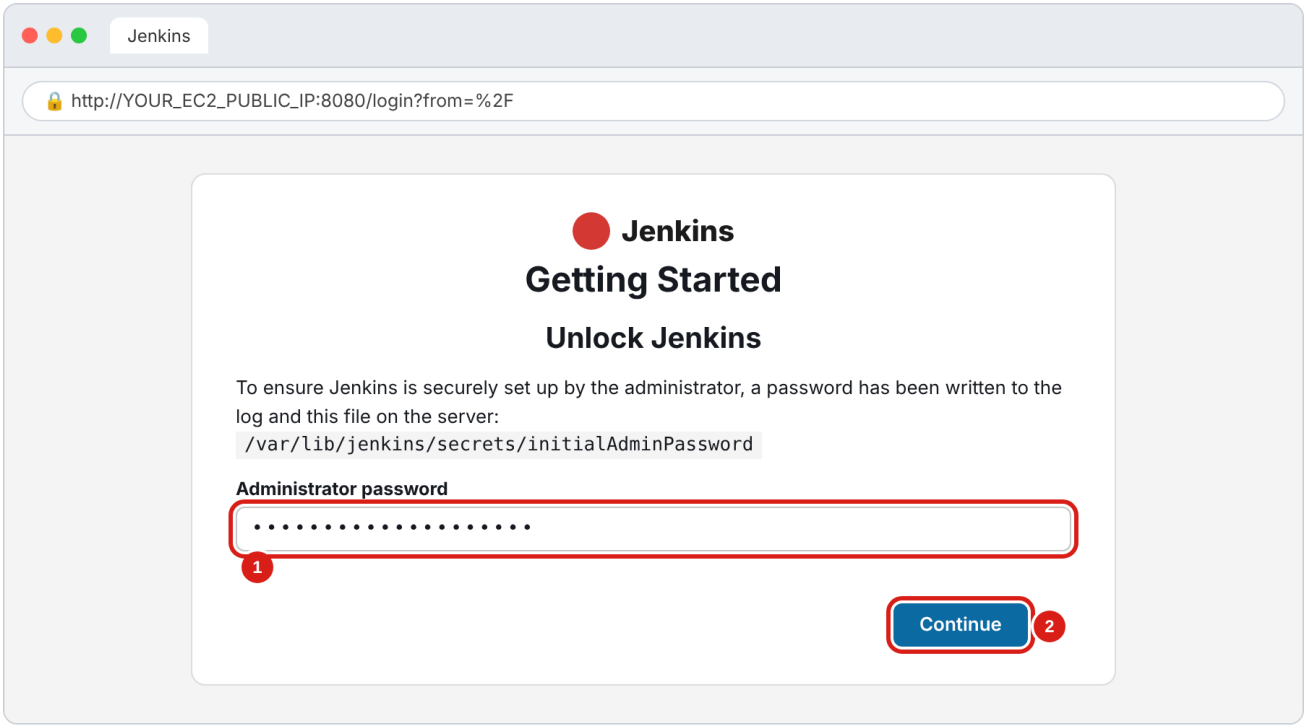


Figure 13. *Unlock Jenkins: paste the administrator password and click Continue* (illustrative screen)

1. Choose **Install suggested plugins** and wait for the installation to finish.
2. Create your admin account.
3. Confirm the Jenkins URL and click **Start using Jenkins**.

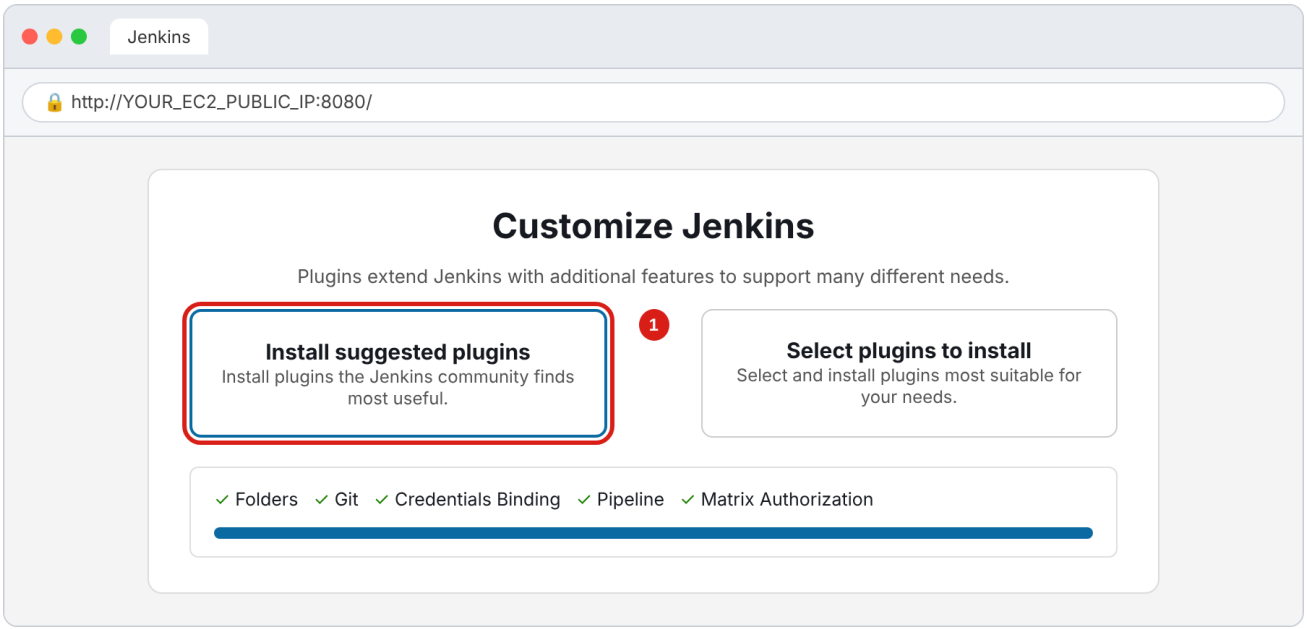


Figure 14. *Customize Jenkins: choose Install suggested plugins* (illustrative screen)

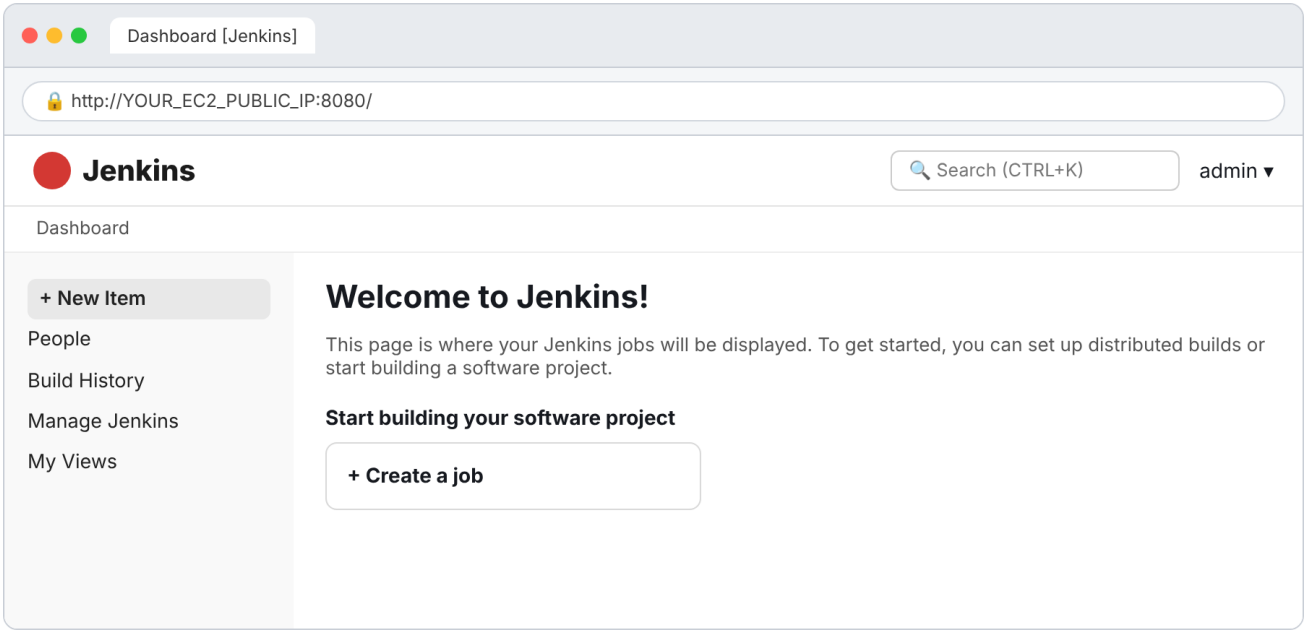


Figure 15. *The Jenkins dashboard after setup is complete* (illustrative screen)

Checkpoint

You now have a working Jenkins dashboard.

PHASE 3 Install Git and Docker on EC2

Steps 10 to 12

STEP 10 Install Git

```
sudo dnf install -y git
git --version
```

Jenkins uses Git to retrieve your website source code from GitHub.

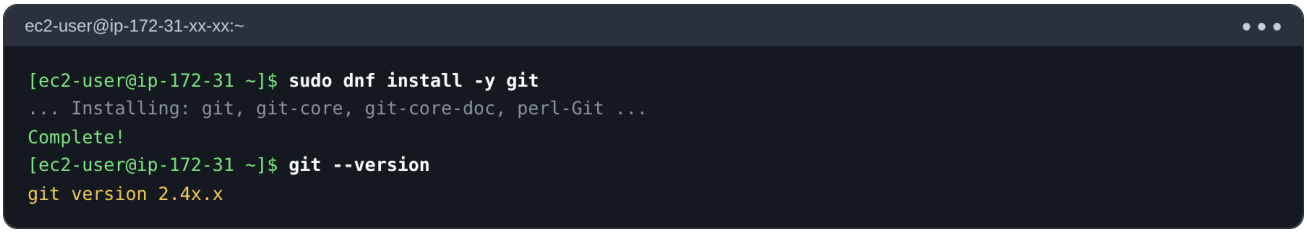


Figure 16. *Git installed and version confirmed* (illustrative screen)

STEP 11 Install Docker Engine

On Amazon Linux 2023:

```
sudo dnf install -y docker
```

Start Docker and enable it at boot:

```
sudo systemctl enable --now docker
sudo systemctl status docker
```

Verify Docker:

```
sudo docker --version
sudo docker ps
```

These commands follow AWS's documented Docker installation approach for Amazon Linux 2023.

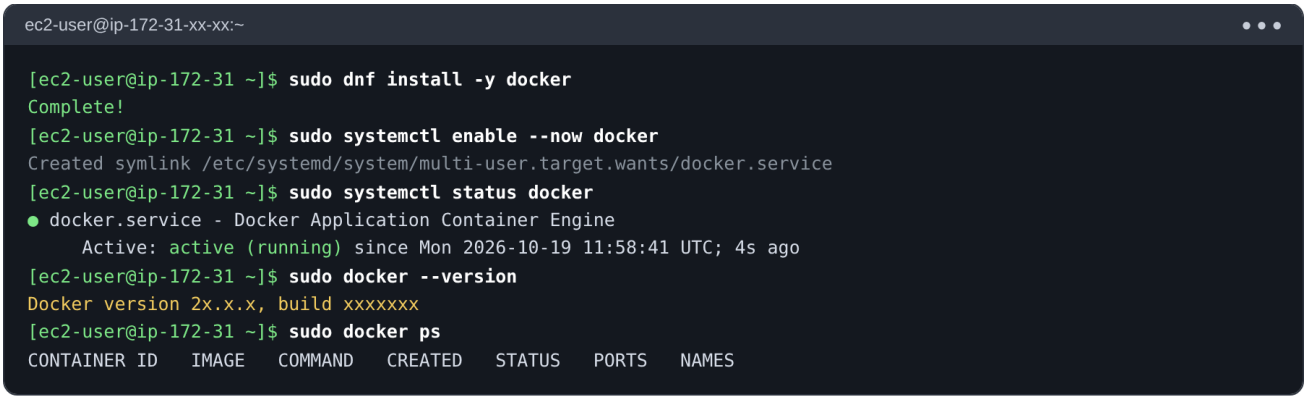


Figure 17. *Docker installed, running and verified with sudo* (illustrative screen)

STEP 12 Allow Jenkins to use Docker

Jenkins runs as its own Linux user, which needs permission to run Docker commands. For this single-instance lab, run:

```
sudo usermod -aG docker jenkins
sudo usermod -aG docker ec2-user
sudo systemctl restart jenkins
```

Reconnect your SSH session so your `ec2-user` group membership takes effect, then check Docker access:

```
docker ps
```

To verify Jenkins can access Docker, check the Jenkins service and build a test job in a later step.

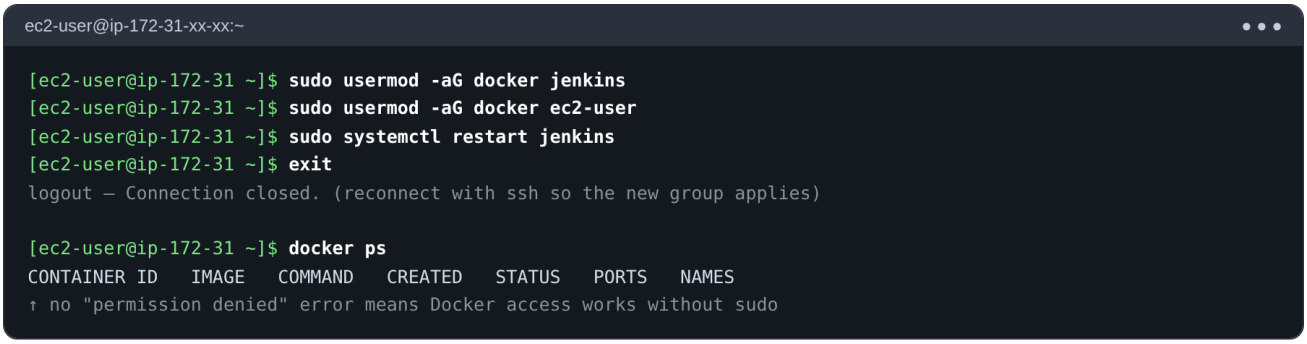


Figure 18. *Adding jenkins and ec2-user to the docker group, then docker ps without sudo* (illustrative screen)

> **Security warning**
>
> Membership in the `docker` group effectively grants root-level control over the host. This shortcut is for a disposable learning lab. In production, use an isolated build agent or a more restrictive container-build architecture.

PHASE 4 Create the GitHub repository

Steps 13 to 17

STEP 13 Create a new repository

1. Open GitHub and click **New repository**.
2. Repository name: `cloud-messenger-netflix-cicd`.
3. Choose **Public** for this beginner lab, so Jenkins can clone it without GitHub credentials.
4. Create the repository.

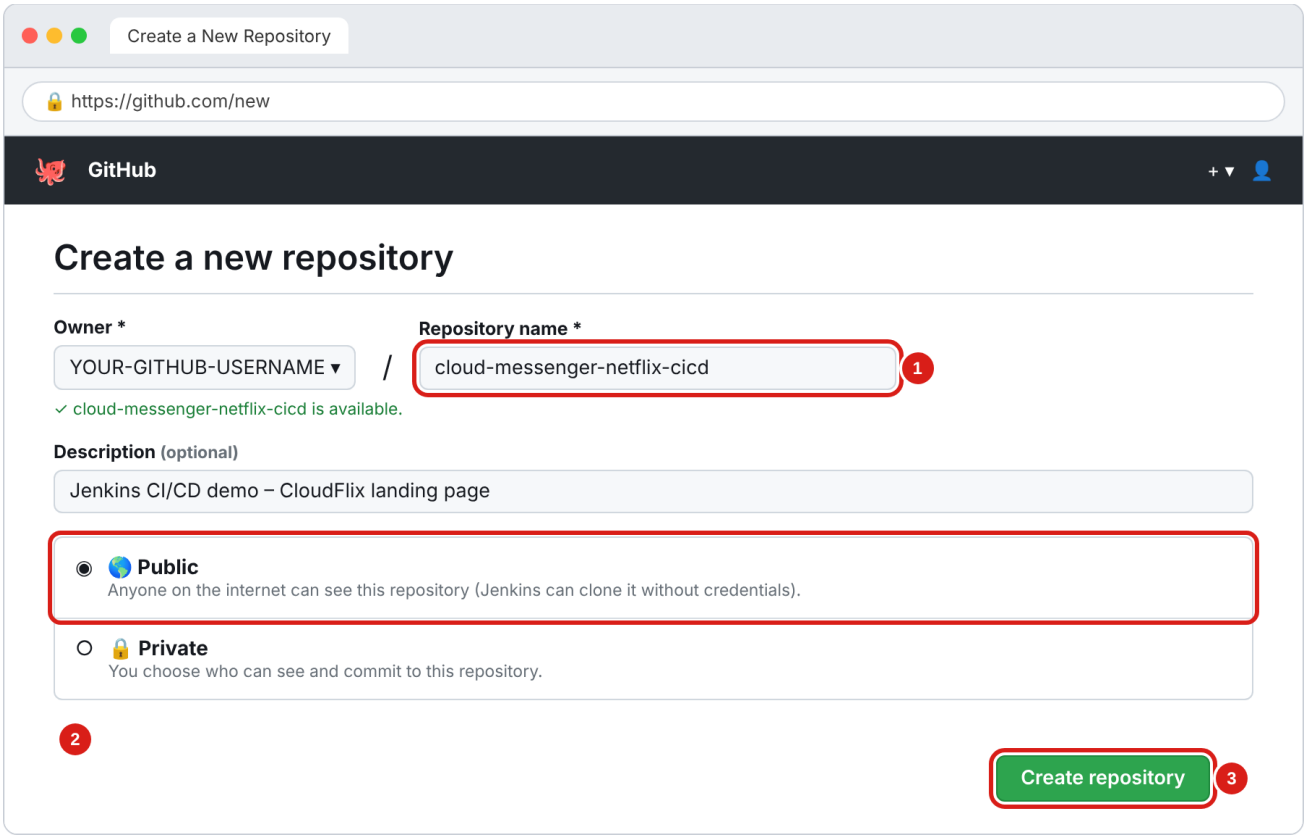


Figure 19. *GitHub: create a public repository named cloud-messenger-netflix-cicd* (illustrative screen)

STEP 14 Prepare your project files

On your computer, create a folder with two files inside:

```
cloud-messenger-netflix-cicd/
  index.html
  Dockerfile
```



Figure 20. *Project folder containing index.html and Dockerfile* (illustrative screen)

STEP 15 Create index.html

Paste the code below into `index.html`. It is a responsive, Netflix-inspired streaming landing page for this educational project.

index.html | Dockerfile

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>CloudFlix | Streaming Demo</title>
 <style>
 * { box-sizing: border-box; }
 body { margin: 0; background: #101010; color: white; }
 /* ... header, .hero, .cards, .card, footer styles ... */
 </style>
</head>
<body>
 <header> <div class="logo">CLOUDFLIX</div> ... </header>
 <section class="hero"> <h1>Stories worth staying up for.</h1> ... </section>
 <main id="shows"> ... four <div class="card"> tiles ... </main>
 <footer>CloudFlix Demo · Built with Git, GitHub, Jenkins and Docker</footer>
</body>
</html>
```

Figure 21. *index.html open in a code editor (abridged view)* (illustrative screen)

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>CloudFlix | Streaming Demo</title>
 <style>
 * { box-sizing: border-box; }
 body {
 margin: 0;
 background: #101010;
 color: white;
 font-family: Arial, sans-serif;
 }
 header {
 display: flex;
 justify-content: space-between;
 align-items: center;
 padding: 22px 6%;
 background: #080808;
 }
 .logo {
 color: #e50914;
 font-size: 30px;
 font-weight: 900;
 letter-spacing: 2px;
 }
 .sign-in {
 background: #e50914;
 padding: 10px 18px;
 border-radius: 5px;
 color: white;
```

```
  text-decoration: none;
  }
  .hero {
  min-height: 420px;
  padding: 90px 7%;
  background:
  linear-gradient(90deg, #101010 10%, #101010d9 55%, #10101055),
  radial-gradient(circle at 75% 30%, #7e1019, #241019 45%, #101010 80%);
  }
  .hero h1 {
  max-width: 650px;
  font-size: clamp(38px, 6vw, 64px);
  margin: 0 0 18px;
  }
  .hero p {
  max-width: 540px;
  font-size: 19px;
  line-height: 1.6;
  color: #ddd;
  }
  .button {
  display: inline-block;
  margin-top: 18px;
  padding: 14px 22px;
  background: #e50914;
  border-radius: 4px;
  color: white;
  text-decoration: none;
  font-weight: bold;
  }
  main { padding: 28px 6%; }
  .cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 16px;
  }
  .card {
  min-height: 150px;
  padding: 22px;
  border-radius: 8px;
  background: linear-gradient(135deg, #4b1020, #202044);
  display: flex;
  align-items: end;
  font-weight: bold;
  font-size: 19px;
  }
  footer {
  padding: 28px;
  text-align: center;
  color: #aaa;
  }
  </style>
</head>
<body>
  <header>
  <div class="logo">CLOUDFLIX</div>
```

```
<a class="sign-in" href="#shows">Explore</a>
</header>
<section class="hero">
<h1>Stories worth staying up for.</h1>
<p>
Explore a world of movies, series, and unforgettable adventures.
Built and deployed through a Jenkins CI/CD pipeline on AWS.
</p>
<a class="button" href="#shows">Explore the collection</a>
</section>
<main id="shows">
<h2>Trending on CloudFlix</h2>
<div class="cards">
<div class="card">The Last Mission</div>
<div class="card">Beyond Earth</div>
<div class="card">Code Red</div>
<div class="card">The Final Chapter</div>
</div>
</main>
<footer>
CloudFlix Demo · Built with Git, GitHub, Jenkins and Docker
</footer>
</body>
</html>
```

> **Keep this headline**
> The pipeline's test stage searches the page for the text `Stories worth staying up for`. If you change that headline later, update the test line in the build script as well.

STEP 16 Create the Dockerfile

Create a file named exactly `Dockerfile` with no extension:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Instruction	Value	Meaning
FROM	`nginx:alpine`	Lightweight Nginx image
COPY	`index.html`	Copies the page into the web root
EXPOSE	`80`	Documents the web port

No separate build tool is needed to package this static website.

index.html Dockerfile

```
1  FROM nginx:alpine (1)
2  COPY index.html /usr/share/nginx/html/index.html (2)
3  EXPOSE 80 (3)
```

(1) lightweight Nginx base image (2) copy the page into the web root (3) documents the web port

Figure 22. *Dockerfile with its three instructions explained* (illustrative screen)

STEP 17 Push the files to GitHub

Open Git Bash or a terminal in the project folder:

```
git init
git add index.html Dockerfile
git commit -m "Create CloudFlix landing page"
git branch -M main
git remote add origin https://github.com/YOUR-GITHUB-USERNAME/cloud-messenger-netflix-cicd.git
git push -u origin main
```

Replace `YOUR-GITHUB-USERNAME` with your GitHub username. If Git asks you to authenticate, follow GitHub's current authentication flow.

Git Bash — cloud-messenger-netflix-cicd

```
$ git init
Initialized empty Git repository in .../cloud-messenger-netflix-cicd/.git/
$ git add index.html Dockerfile
$ git commit -m "Create CloudFlix landing page"
[master (root-commit) 4be91a7] Create CloudFlix landing page
 2 files changed, 98 insertions(+)
 create mode 100644 Dockerfile
 create mode 100644 index.html
$ git branch -M main
$ git remote add origin https://github.com/YOUR-GITHUB-USERNAME/cloud-messenger-netflix-cicd.git
$ git push -u origin main
Enumerating objects: 4, done.
Writing objects: 100% (4/4), 1.62 KiB | 1.62 MiB/s, done.
To https://github.com/YOUR-GITHUB-USERNAME/cloud-messenger-netflix-cicd.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Figure 23. *Initialising the repository and pushing to GitHub* (illustrative screen)

Verify that both files are visible in the repository.

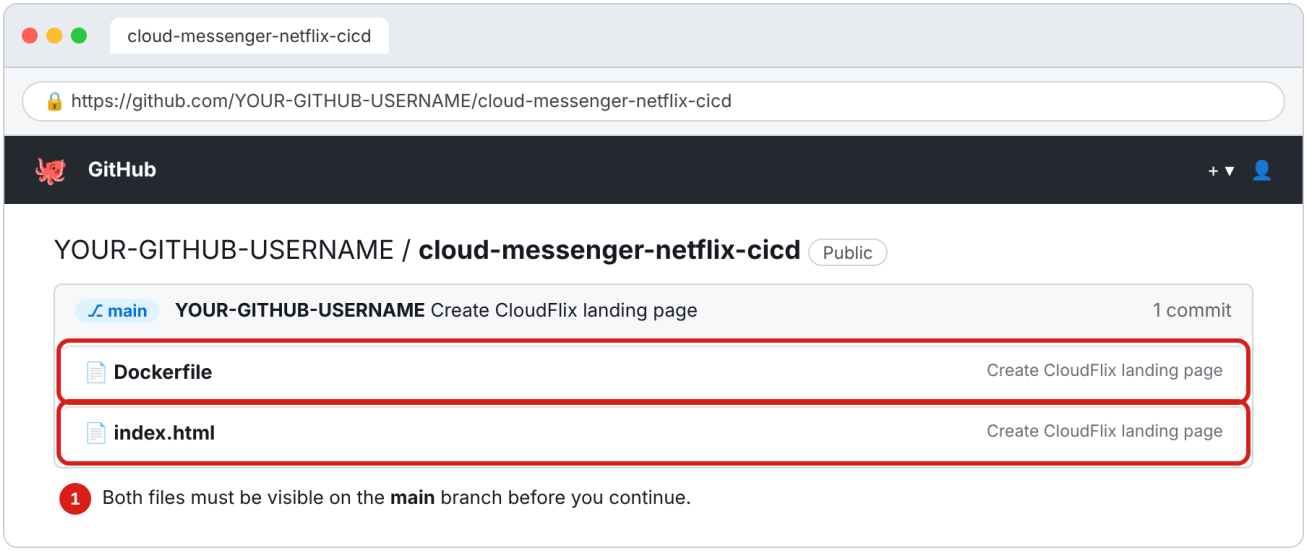


Figure 24. *Both files visible on the main branch in GitHub* (illustrative screen)

PHASE 5 Prepare Docker Hub

Steps 18 to 19

STEP 18 Create your Docker Hub repository

1. Open Docker Hub and sign in or create an account.
2. Select **My Hub**, then **Repositories**, then **Create repository**.
3. Repository name: `cloud-messenger-netflix`.
4. Set visibility to **Public** for this lab, and create the repository.

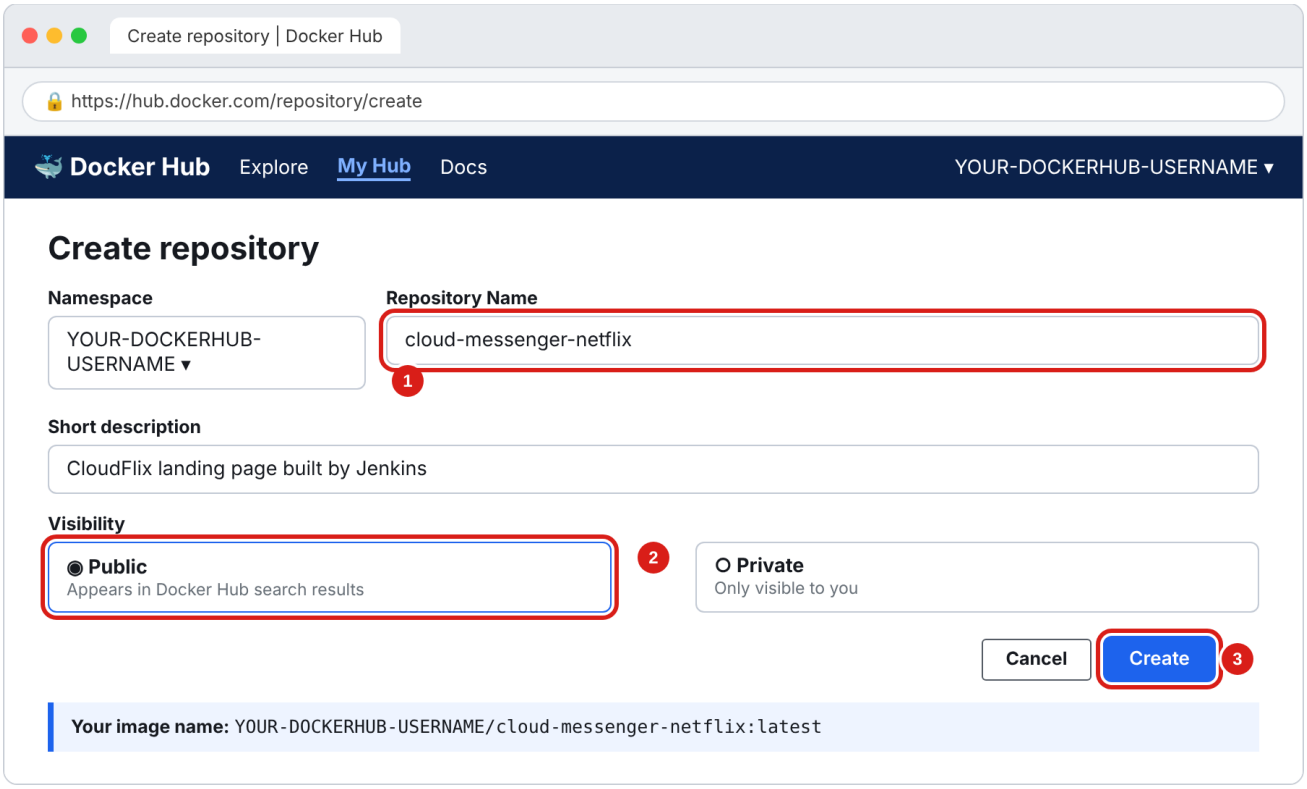


Figure 25. *Docker Hub: create a public repository named cloud-messenger-netflix* (illustrative screen)

Your image name will be:

```
YOUR-DOCKERHUB-USERNAME/cloud-messenger-netflix:latest
```

STEP 19 Create a Docker Hub access token

Use an access token rather than your Docker Hub account password.

1. Open Docker Hub account settings and go to **Personal access tokens**.
2. Create a token with the minimum permissions needed to push to your repository.
3. Copy the token and store it securely.

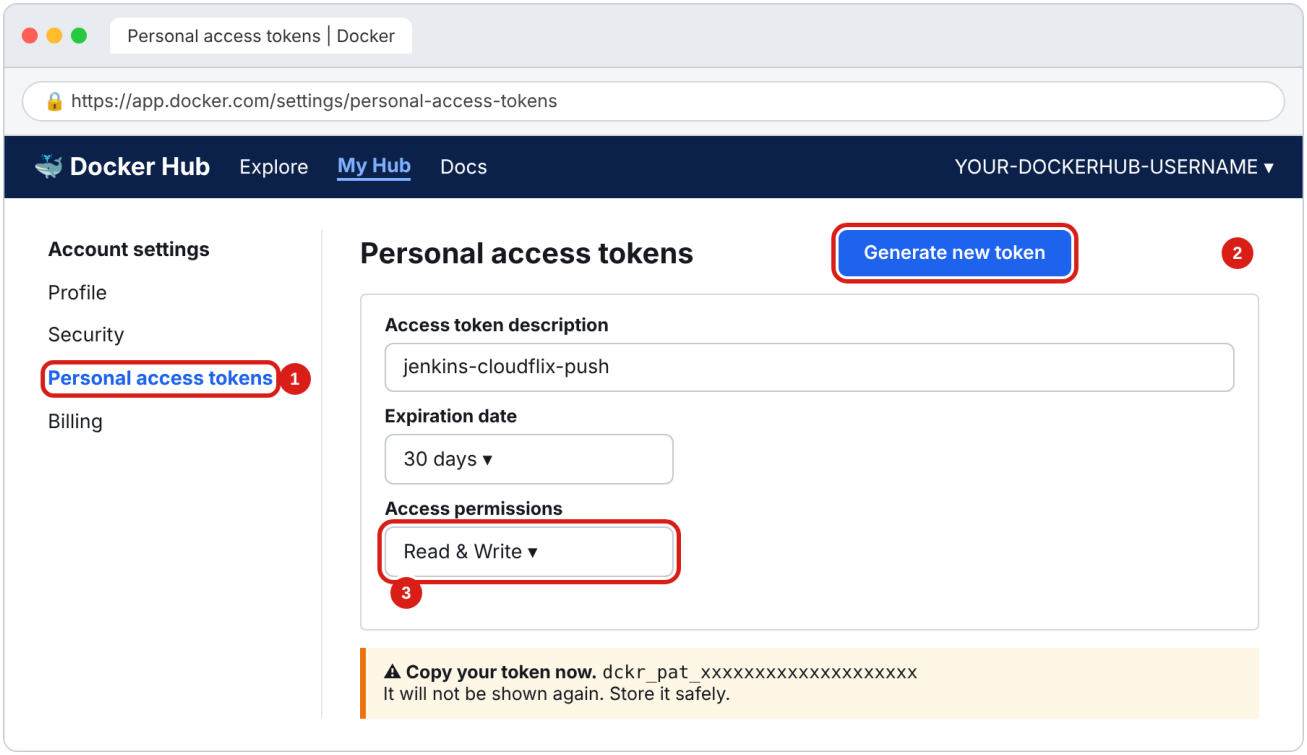


Figure 26. *Generating a personal access token for Jenkins* (illustrative screen)

> **Keep it secret**
> You will use this token in Jenkins credentials. Do not put it in your GitHub repository, Dockerfile or shell script.

PHASE 6 Configure Jenkins credentials and plugins

Steps 20 to 21

STEP 20 Install the required plugins

1. From the dashboard, click **Manage Jenkins**, then **Plugins**, then **Available plugins**.
2. Search for and install the **Git plugin** and the **Credentials Binding plugin**.
3. Restart Jenkins if prompted.

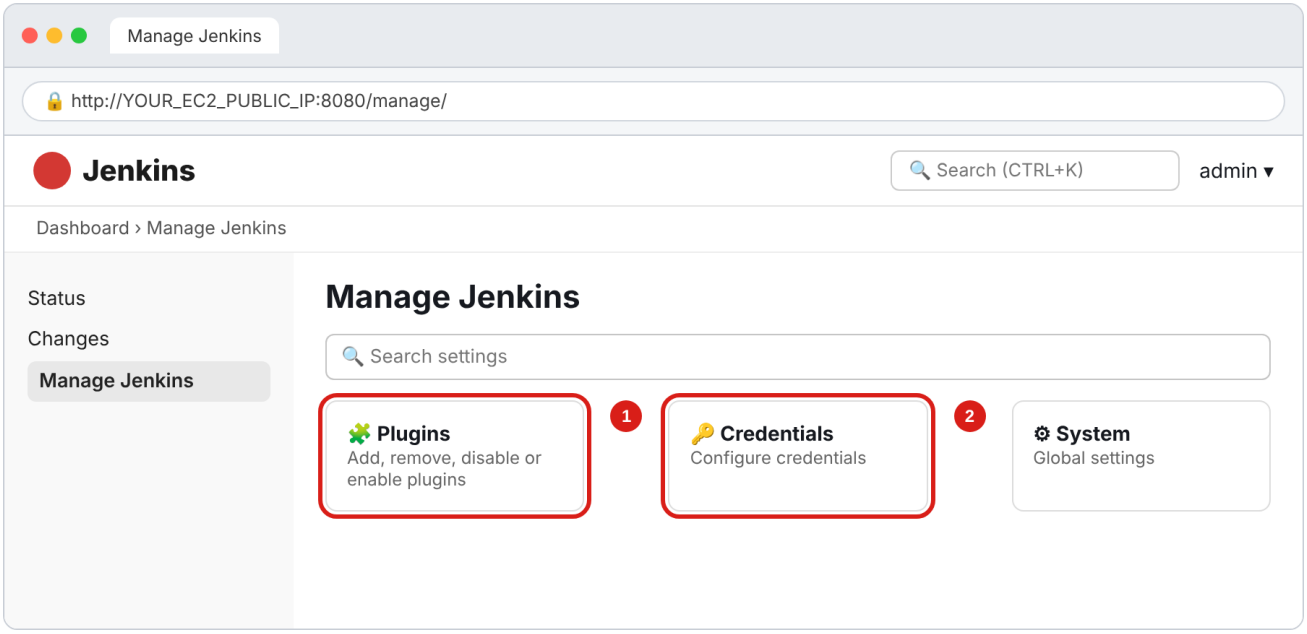


Figure 27. *Manage Jenkins: open Plugins (1) now, and Credentials (2) in the next step* (illustrative screen)

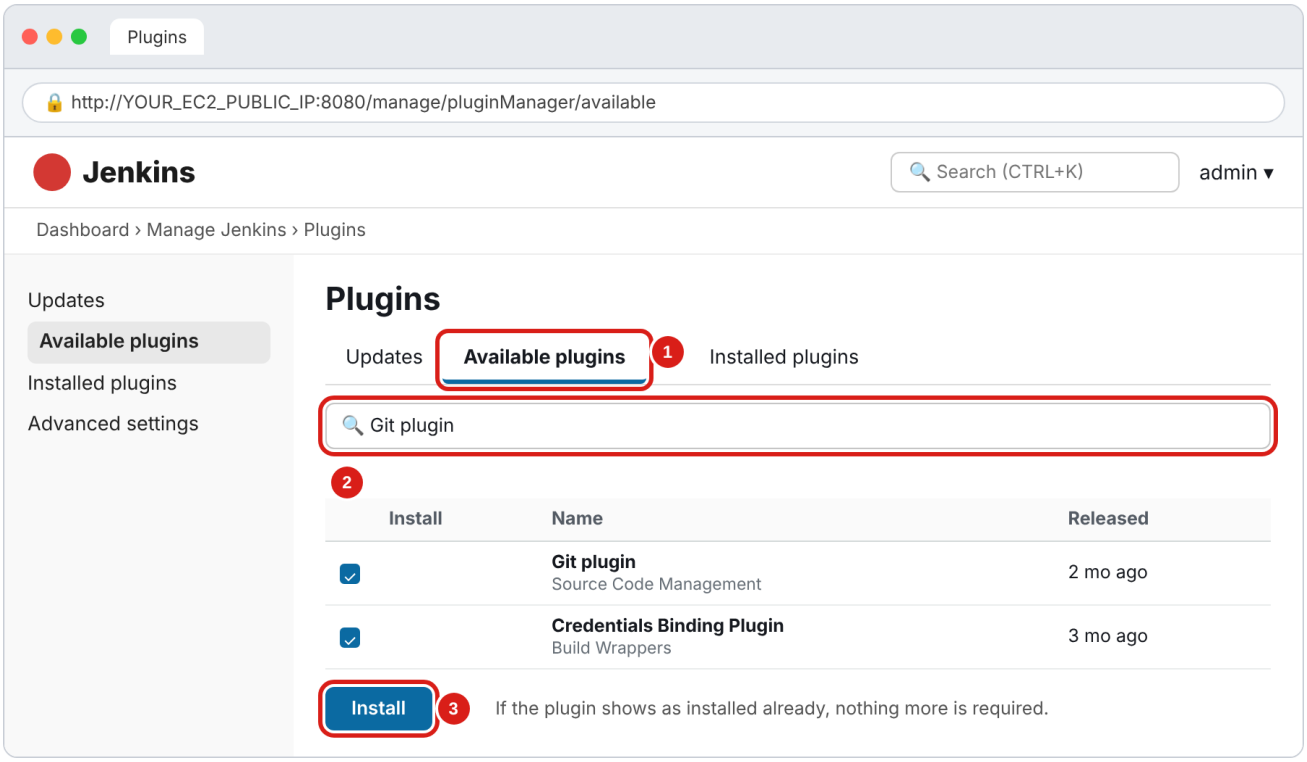


Figure 28. *Available plugins: select the Git plugin and Credentials Binding plugin, then Install* (illustrative screen)

The Git plugin lets Jenkins check out your repository. Credentials Binding lets a build use stored secrets without hardcoding them into the project.

STEP 21 Add Docker Hub credentials

1. Go to **Manage Jenkins**, then **Credentials**.
2. Open **System**, then **Global credentials (unrestricted)**.
3. Click **Add Credentials** and choose **Username with password**.
4. Enter the values below, then click **Create**.

Field	Value
Username	Your Docker Hub username
Password	Your Docker Hub access token
ID	`dockerhub-creds`
Description	Docker Hub push credentials

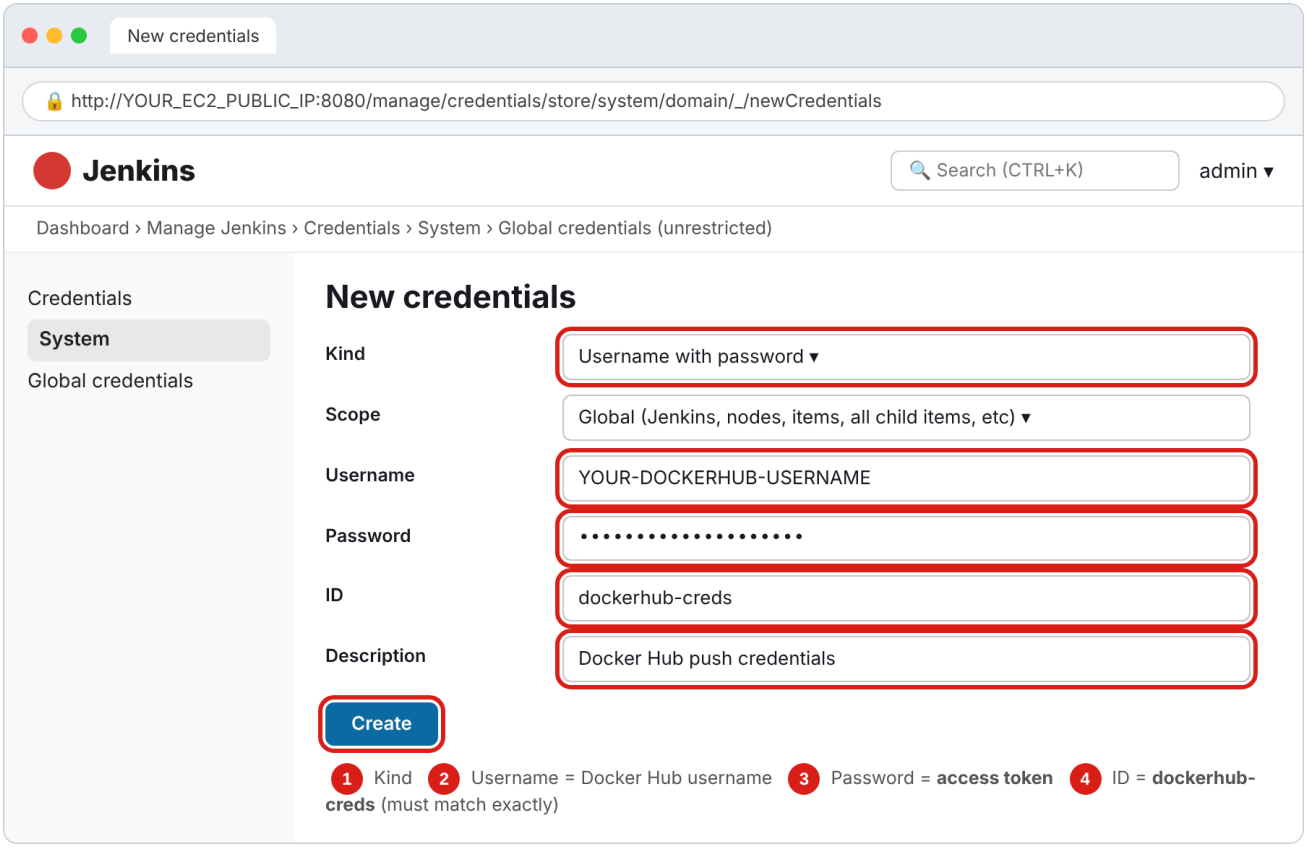


Figure 29. *New credentials form for Docker Hub* (illustrative screen)

Jenkins now stores the credentials so the job can access them securely.

PHASE 7 Create the Jenkins Freestyle CI/CD job

Steps 22 to 25

STEP 22 Create a new Freestyle project

1. From the dashboard, click **New Item**.
2. Enter the job name `CloudFlix-CICD-Pipeline`.
3. Select **Freestyle project** and click **OK**.

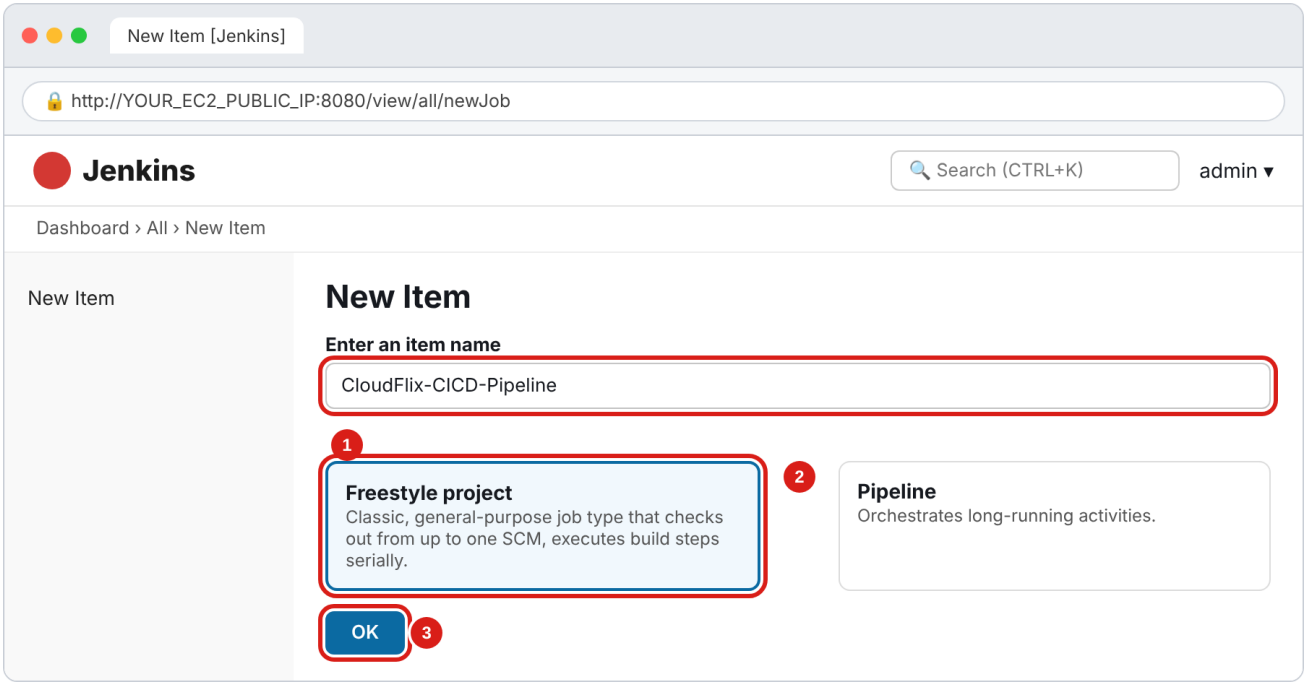


Figure 30. *New Item: name the job and choose Freestyle project* (illustrative screen)

STEP 23 Configure General settings

Under **General**, add this description:

```
CI/CD pipeline to checkout CloudFlix source code from GitHub,
build and test a Docker image, push it to Docker Hub, and
deploy the container on AWS EC2.
```

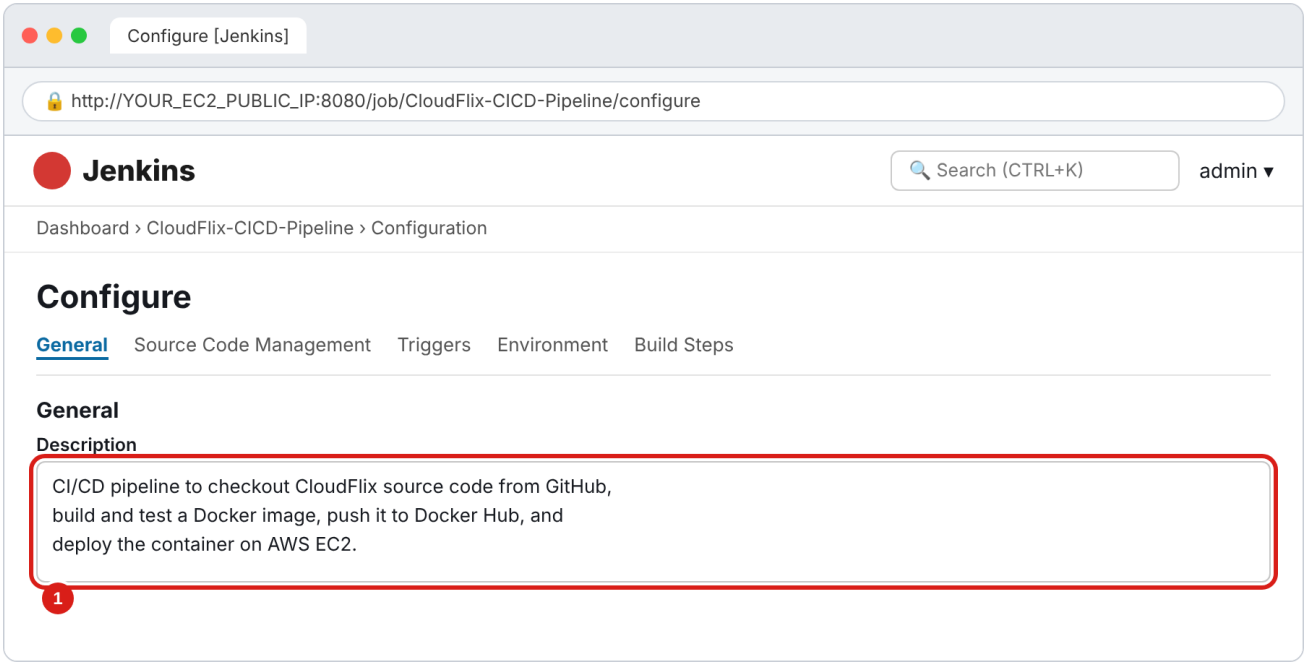


Figure 31. *General section with the job description* (illustrative screen)

STEP 24 Connect Jenkins to GitHub

Under **Source Code Management**, select **Git** and fill in:

Field	Value
Repository URL	`https://github.com/YOUR-GITHUB-USERNAME/cloud-messenger-netflix-cicd.git`
Credentials	None, because the repository is public
Branch Specifier	`*/main`

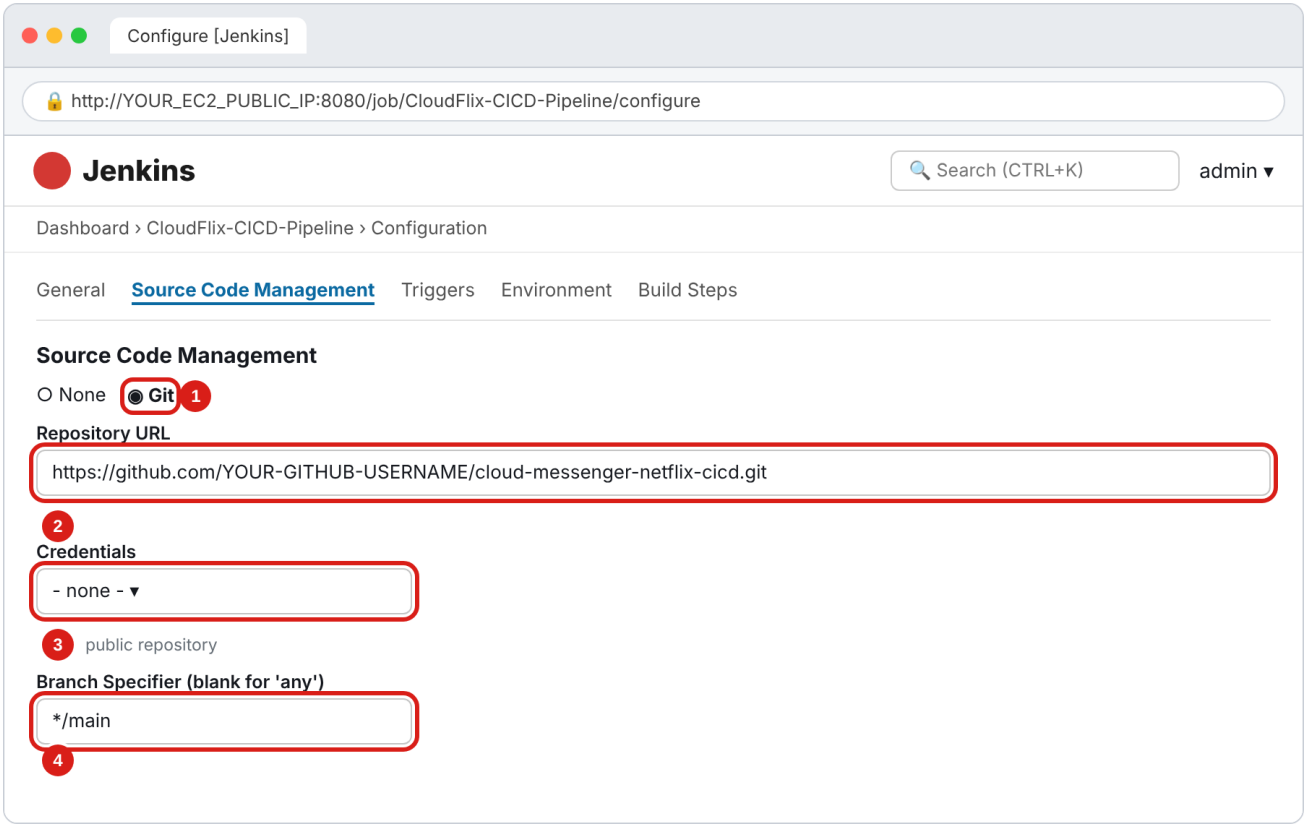


Figure 32. *Source Code Management: Git repository URL and branch* (illustrative screen)

Jenkins clones the source code into its workspace whenever the job runs.

STEP 25 Bind Docker Hub credentials

Scroll to **Build Environment**:

1. Enable **Use secret text(s) or file(s)**.
2. Add a binding of type **Username and password (separated)**.
3. Select credential `dockerhub-creds`.
4. Username variable: `DOCKERHUB_USER`.
5. Password variable: `DOCKERHUB_TOKEN`.

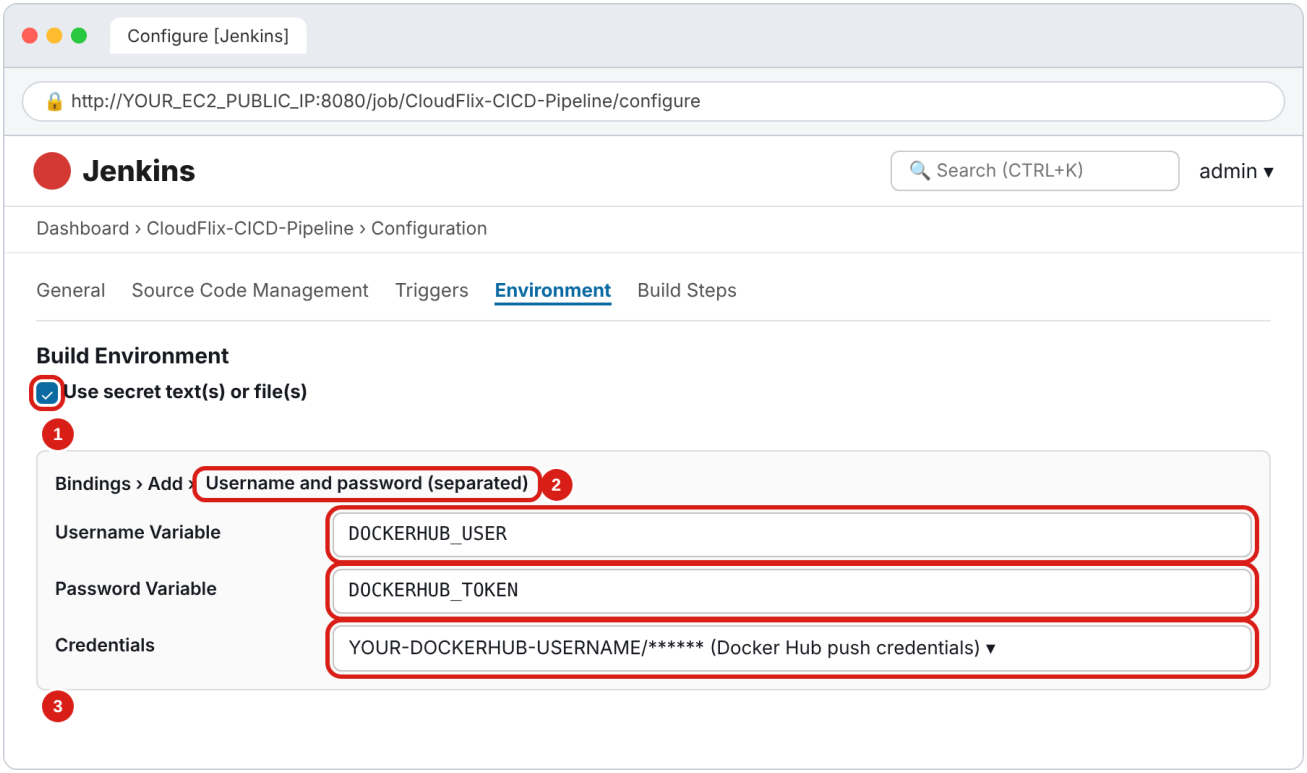


Figure 33. *Build Environment: bind dockerhub-creds to DOCKERHUB_USER and DOCKERHUB_TOKEN* (illustrative screen)

> **Option missing?**
> Verify that the Credentials Binding plugin is installed and that Jenkins has reloaded it.

PHASE 8 Add the CI/CD build script

Steps 26 to 27

STEP 26 Add the build step

1. Under **Build Steps**, click **Add build step** and select **Execute shell**.
2. Paste the following script.

```
#!/bin/bash
set -eu

echo "==================================="
echo "STEP 1: Verify source code"
echo "==================================="
test -s index.html
test -s Dockerfile
echo "Website source files found."

echo "==================================="
echo "STEP 2: Build Docker image"
echo "==================================="
IMAGE="$DOCKERHUB_USER/cloud-messenger-netflix:latest"
docker build --pull -t "$IMAGE" .
echo "Docker image built successfully."

echo "==================================="
echo "STEP 3: Test the Docker image"
echo "==================================="
docker run -d --rm \
 --name cloudflix-test \
 -p 127.0.0.1:18081:80 \
 "$IMAGE"
trap 'docker stop cloudflix-test >/dev/null 2>&1 || true' EXIT
sleep 3
curl -fsS http://127.0.0.1:18081/ -o /tmp/cloudflix-test.html
grep -q "Stories worth staying up for" /tmp/cloudflix-test.html
echo "Website test passed."
docker stop cloudflix-test
trap - EXIT

echo "==================================="
echo "STEP 4: Login to Docker Hub"
echo "==================================="
printf '%s' "$DOCKERHUB_TOKEN" |
 docker login --username "$DOCKERHUB_USER" --password-stdin
echo "Docker Hub login successful."

echo "==================================="
echo "STEP 5: Push image to Docker Hub"
echo "==================================="
docker push "$IMAGE"
docker logout
echo "Image pushed successfully."

echo "==================================="
```

```
echo "STEP 6: Deploy container"
echo "==================================="
docker rm -f cloudflix || true
docker run -d \
 --name cloudflix \
 --restart unless-stopped \
 -p 8081:80 \
 "$IMAGE"

echo "==================================="
echo "CI/CD PIPELINE COMPLETED"
echo "==================================="
docker ps --filter name=cloudflix
```

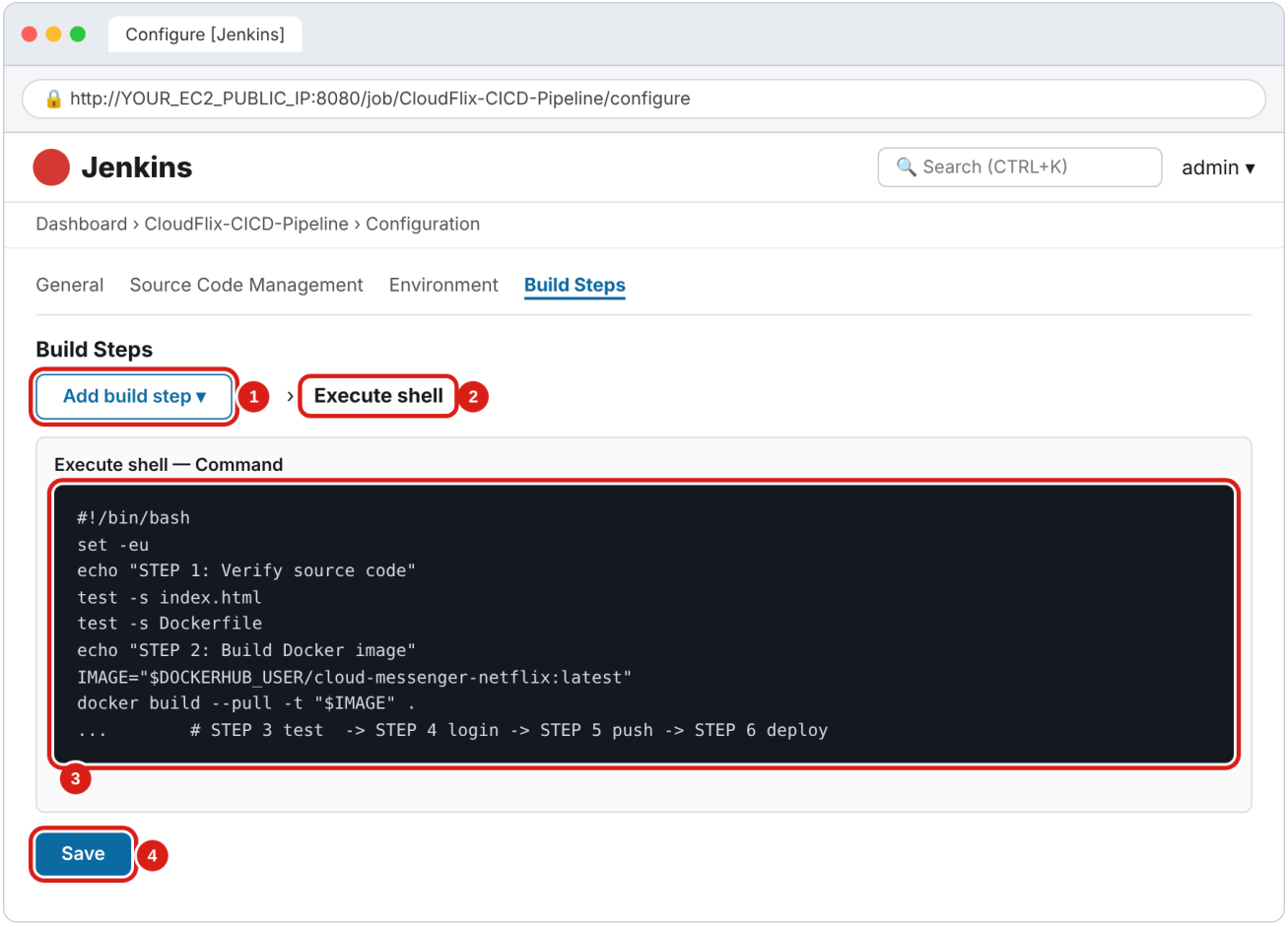


Figure 34. *Execute shell build step with the pipeline script (abridged view)* (illustrative screen)

Before running

Ensure `curl` is installed (`sudo dnf install -y curl`), the Docker service is running, and the Jenkins service has Docker access.

The script performs six stages:

#	Stage	What it does
1	Verify source	Validates that `index.html` and `Dockerfile` exist and are not empty

#	Stage	What it does
2	Build image	Builds the Docker image tagged `$DOCKERHUB_USER/cloud-messenger-netflix:latest`
3	Test image	Runs a temporary container on 127.0.0.1:18081 and checks the website content
4	Docker Hub login	Authenticates with the token supplied through `--password-stdin`
5	Push image	Pushes the image to Docker Hub, then logs out
6	Deploy	Replaces the `cloudflix` container and starts it on port 8081

STEP 27 Save the job

Click **Save** (badge 4 in the previous figure). Your first end-to-end Freestyle CI/CD job is now configured and you land on the job page, which has a **Build Now** button in the left menu.

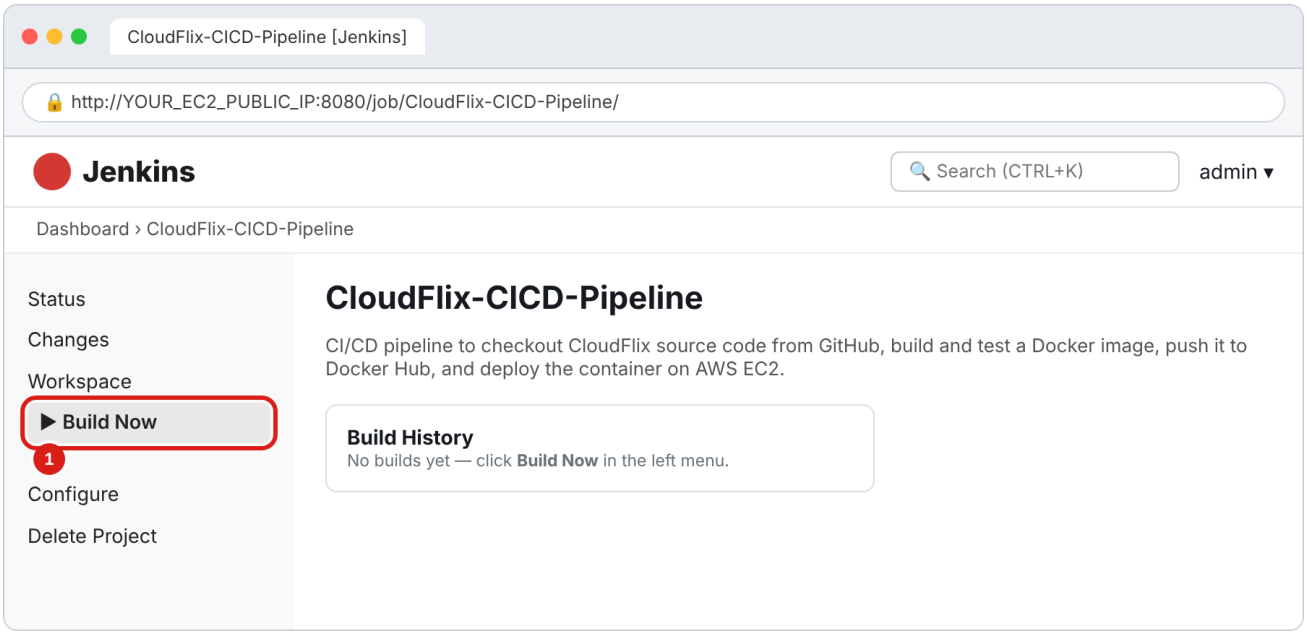


Figure 35. *Job page after saving: Build Now is in the left menu* (illustrative screen)

PHASE 9 Run and verify the pipeline

Steps 28 to 30

STEP 28 Execute the job

1. Open `CloudFlix-CICD-Pipeline` and click **Build Now**.
2. Wait for the build to finish.
3. Under **Build History**, click the latest build number, then **Console Output**.

A successful run should finish with:

```
Website source files found.
Docker image built successfully.
Website test passed.
Docker Hub login successful.
Image pushed successfully.
CI/CD PIPELINE COMPLETED
Finished: SUCCESS
```

The exact output will contain additional Docker and Jenkins logs.

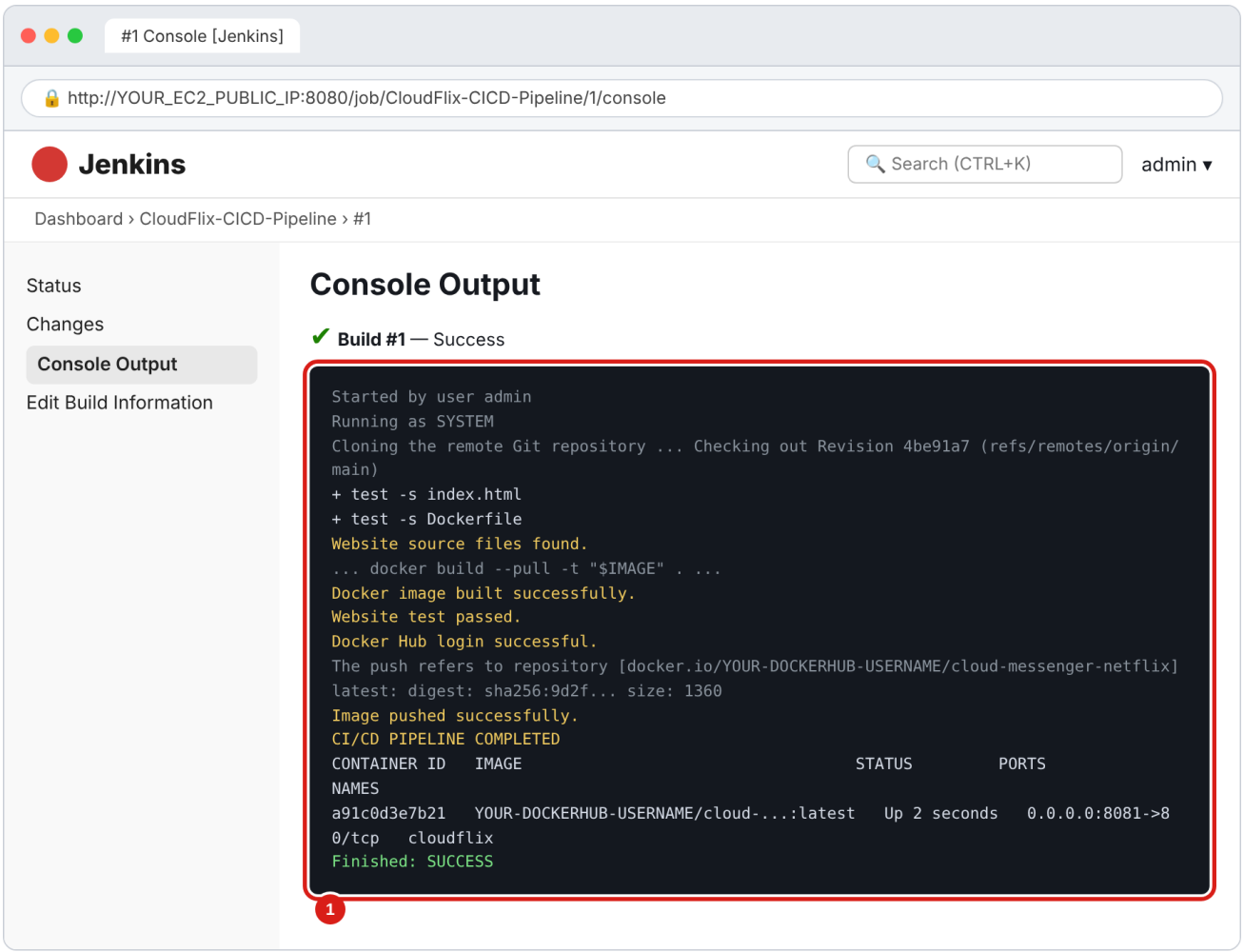


Figure 36. *Console Output of build #1 ending in Finished: SUCCESS* (illustrative screen)

STEP 29 Verify the image in Docker Hub

Open Docker Hub, go to **Repositories**, open `cloud-messenger-netflix` and confirm that the `latest` tag exists.

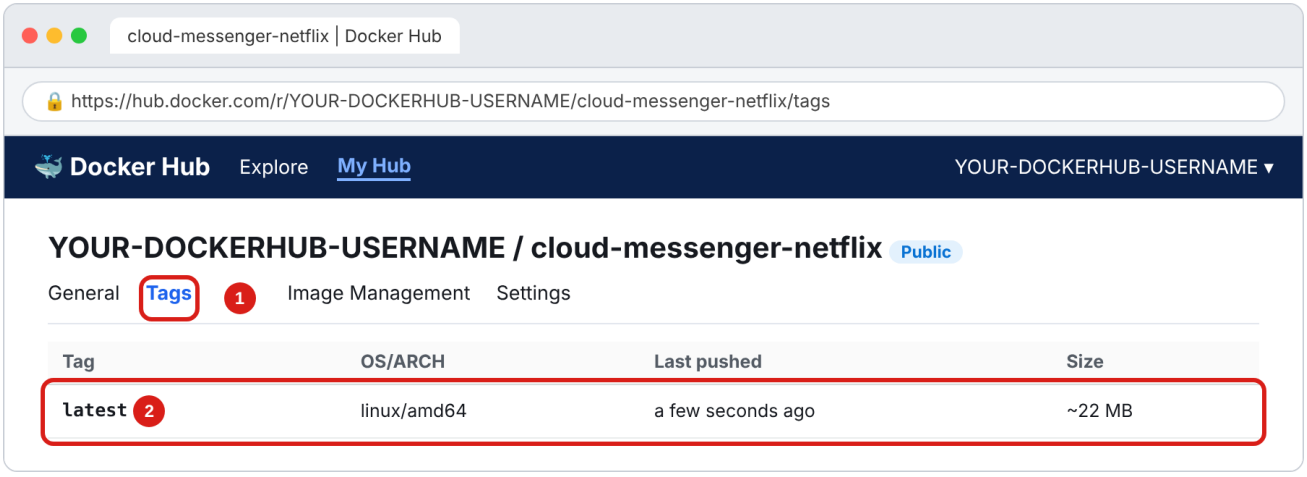


Figure 37. *The latest tag in the Docker Hub repository* (illustrative screen)

STEP 30 View the website

Open your browser at `http://YOUR_EC2_PUBLIC_IP:8081`. You should see your CloudFlix landing page.

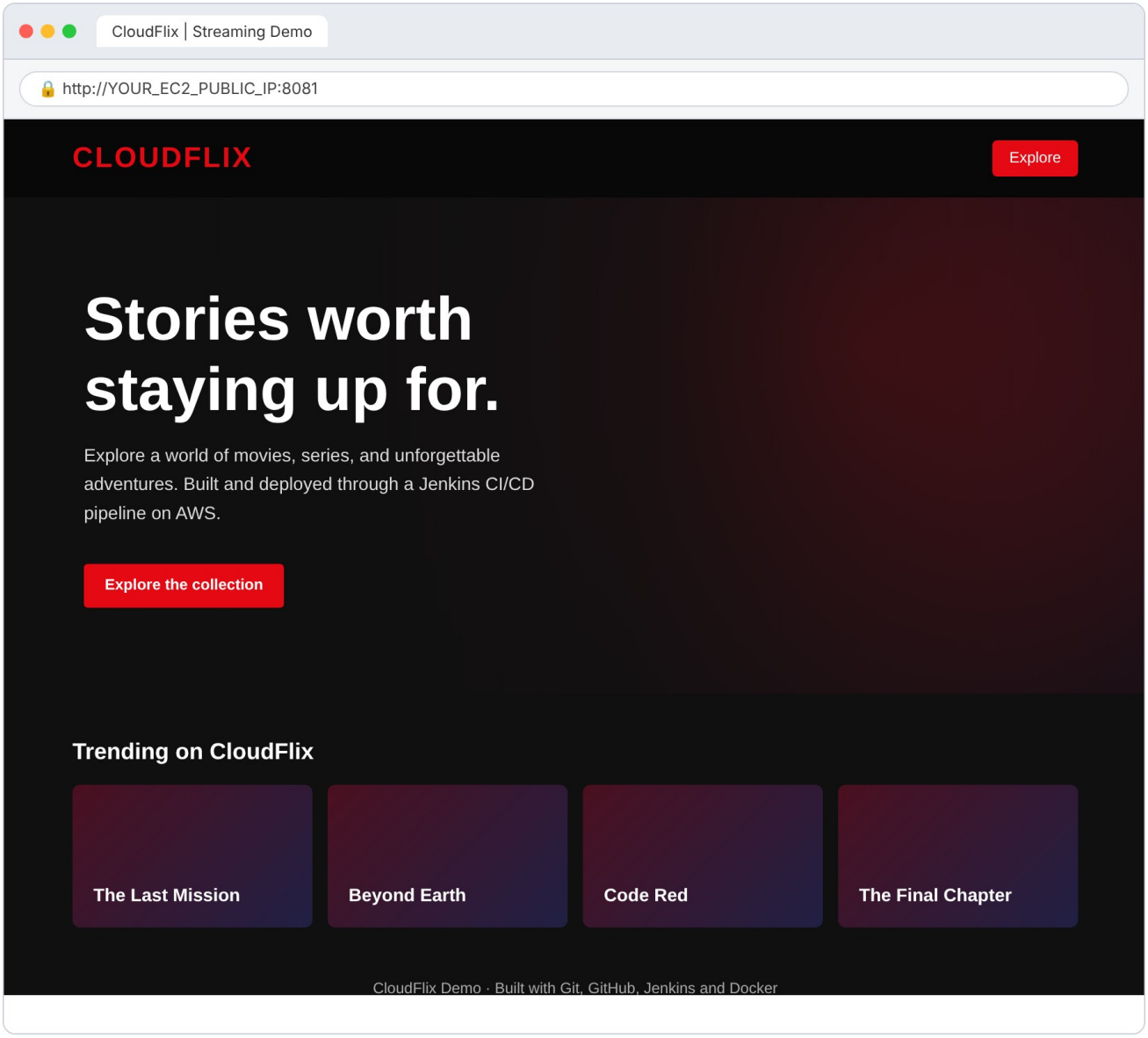


Figure 38. *The CloudFlix landing page served from the container on port 8081*

> **Browser can't connect?**
> Verify that TCP port 8081 is allowed in your EC2 security group from your IP address and that the container is running:

```
docker ps
docker logs cloudflix
curl -I http://localhost:8081
```

```
ec2-user@ip-172-31-xx-xx:~
[ec2-user@ip-172-31 ~]$ docker ps
CONTAINER ID   IMAGE                      STATUS          PORTS                   NAMES
a91c0d3e7b21   .../cloud-messenger-...    Up 3 minutes    0.0.0.0:8081->80/tcp    cloudflix

[ec2-user@ip-172-31 ~]$ docker logs cloudflix
/docker-entrypoint.sh: Configuration complete; ready for start up

[ec2-user@ip-172-31 ~]$ curl -I http://localhost:8081
HTTP/1.1 200 OK
Server: nginx/1.2x.x
Content-Type: text/html
```

Figure 39. *Healthy container: running, clean logs and an HTTP 200 response* (illustrative screen)

PHASE 10 Test automatic CI/CD behaviour

The change-and-rebuild loop

Your first run is manual. To demonstrate continuous integration, make a small change to `index.html` on your local machine.

1. Change a line of text in `index.html`, then commit and push it to GitHub.
2. In Jenkins, open the job and click **Build Now** again.
3. Confirm Jenkins checks out the updated code, rebuilds the image, pushes it to Docker Hub, and redeploys the container.

```
git add index.html
git commit -m "Update CloudFlix landing page"
git push origin main
```

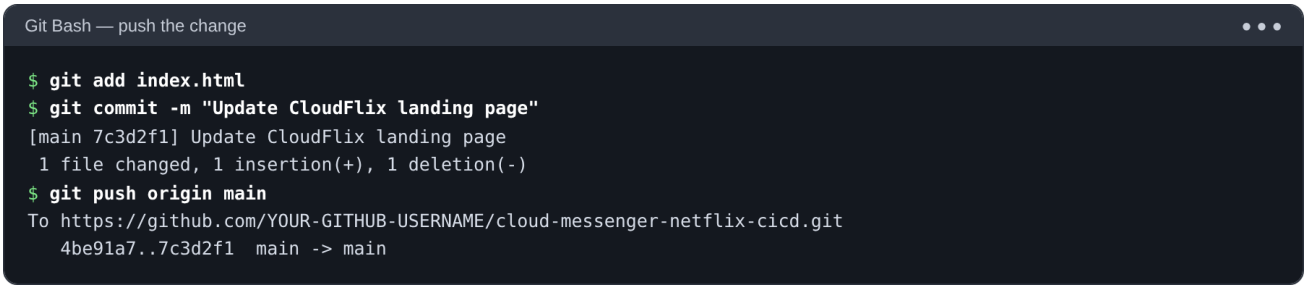


Figure 40. *Committing and pushing a change to GitHub* (illustrative screen)

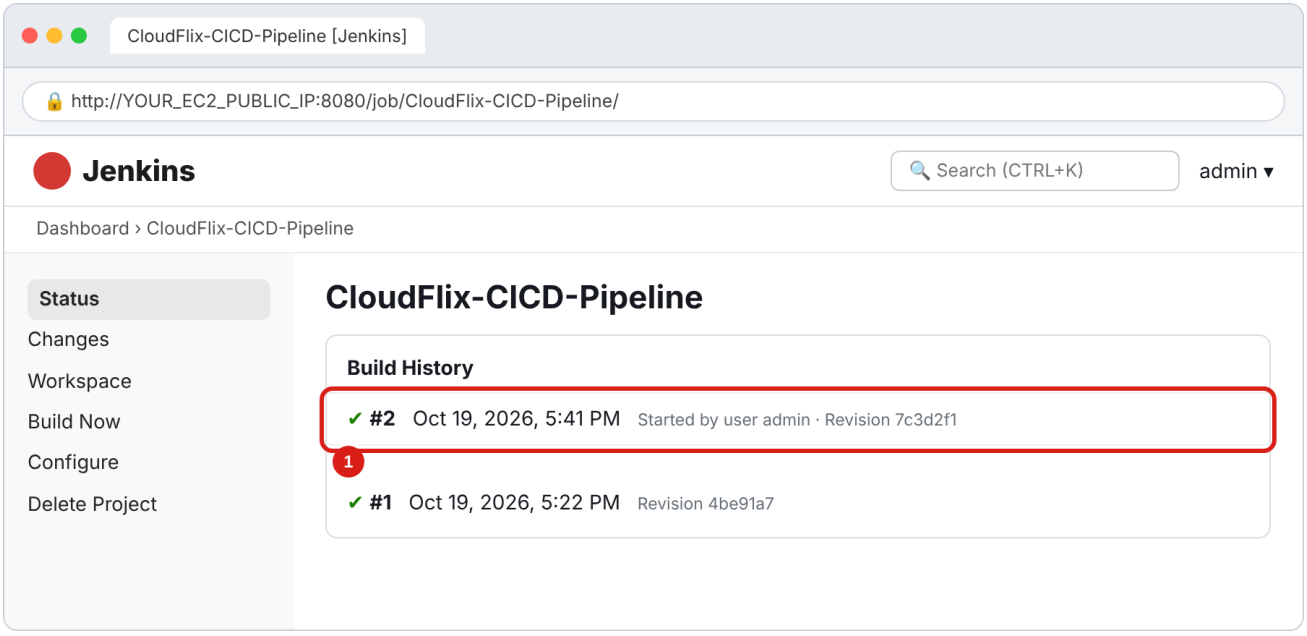


Figure 41. *Build #2 runs against the new commit and succeeds* (illustrative screen)

Tip

Change a card title or the footer text rather than the headline `Stories worth staying up for`, which the build's test stage checks. Refresh `http://YOUR_EC2_PUBLIC_IP:8081` to see your change.

Want builds on every push?

Configure a GitHub webhook and the appropriate Jenkins trigger. Manual builds are sufficient for the initial lab.

Final architecture

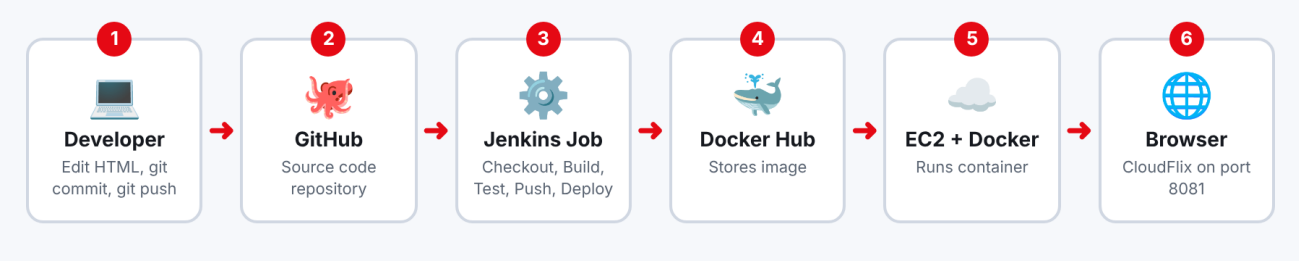


Figure 42. *Final architecture: from git push to the live container*

Important production note

This single-instance design is for learning. Jenkins and the application share the same EC2 host, so it is not a production-grade deployment architecture. For production, use a dedicated Jenkins controller and isolated build agents, secure Jenkins with HTTPS and restricted access, and deploy the container to a separate target such as ECS or EKS.

Troubleshooting quick reference

Symptom	Likely cause	What to do
Jenkins page does not open on port 8080	Port 8080 not allowed from your IP, your IP changed, or Jenkins is stopped	Check the `Jenkins-CICD-SG` rule and run `sudo systemctl status jenkins`
Binding type Username and password (separated) is missing	Credentials Binding plugin not installed or not reloaded	Install it under Manage Jenkins, Plugins, then restart Jenkins
`permission denied` on the Docker socket during a build	The `jenkins` user is not in the `docker` group, or Jenkins was not restarted	Repeat Step 12, including `sudo systemctl restart jenkins`
`curl: command not found` in the build	curl is not installed	`sudo dnf install -y curl`
Build fails at Stage 3 (grep)	The test text was changed in `index.html`	Restore `Stories worth staying up for` or edit the test line
Stage 3 fails because the name or port is in use	A previous test container was left running	`docker rm -f cloudflix-test`, then rebuild
Push is denied by Docker Hub	Wrong username, wrong repository name, or a token without write access	Recheck the credential and create a token with push permission
Browser can't connect on port 8081	Port 8081 blocked or the container is not running	Allow 8081 from your IP, then run `docker ps`, `docker logs cloudflix`, `curl -I http://localhost:8081`

Cleaning up after the lab

This is a disposable learning environment. When you are finished:

- **Stop or terminate** the EC2 instance so it stops incurring charges (terminate to also remove its storage).
- **Delete the Docker Hub access token** you created in Step 19.
- Remove the `Jenkins-CICD-SG` security group and the key pair if you no longer need them.

Appendix A. Names, values and placeholders

Names used in this lab

Item	Value
EC2 instance name	`CloudMessenger-Jenkins-CICD`
AMI and size	Amazon Linux 2023, `t3.medium`, 20 to 30 GiB
Security group	`Jenkins-CICD-SG` (ports 22, 8080, 8081 from My IP)
GitHub repository	`cloud-messenger-netflix-cicd`, branch `main`
Docker Hub repository	`cloud-messenger-netflix`
Image name	`YOUR-DOCKERHUB-USERNAME/cloud-messenger-netflix:latest`
Jenkins credential ID	`dockerhub-creds`
Binding variables	`DOCKERHUB_USER`, `DOCKERHUB_TOKEN`
Jenkins job	`CloudFlix-CICD-Pipeline`
Deployed container	`cloudflix` (host port 8081 to container port 80)
Temporary test container	`cloudflix-test` (127.0.0.1:18081 to container port 80)

Placeholders to replace

Placeholder	Replace with
`YOUR_EC2_PUBLIC_IP`	The Public IPv4 address of your EC2 instance
`your-key.pem`	The key pair file you selected when launching the instance
`YOUR-GITHUB-USERNAME`	Your GitHub username
`YOUR-DOCKERHUB-USERNAME`	Your Docker Hub username

Official references

- Jenkins on AWS
- Docker installation on Linux
- Jenkins credentials